

Modeling Energy Storage in Co-Simulation Testbeds for Data Centers

Bachelor's Thesis — Paul Michael Kilian (0456866)

28. September 2026

First Reviewer: Prof. Dr. habil. Odej Kao
Second Reviewer: Prof. Dr. rer. nat. Volker Markl
Supervisor: Philipp Wiesner



Distributed and Operating Systems
Department of Telecommunication Systems
Faculty IV – Electrical Engineering and Computer Science
Technische Universität Berlin / Technical University of Berlin

Eidesstattliche Erklärung/ Statutory Declaration

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und eigenhändig sowie ohne unerlaubte fremde Hilfe und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe.

I hereby declare that I have completed this thesis independently, single-handedly and without any disallowed external help. This work was made using exclusively the sources and aids listed.

Abstract

In Zeiten steigender Nachfrage nach Rechenleistung führt die Forderung für mehr Nachhaltigkeit im Energiebereich zu Veränderungen der Infrastruktur von Rechenzentren hinsichtlich des Stromversorgungssystems. Viele Anbieter entscheiden sich nun für den Betrieb ihrer Rechenanlagen in Microgrid Umgebungen, da dieser Ansatz bessere Kontrolle und Flexibilität bietet, die eine effizientere Regulierung des Energieverbrauchs und eine Anpassung an die schwankenden Stromproduktionen der erneuerbaren Energiequellen ermöglicht. Die Notwendigkeit, Fortschritte bei der Steuerung solch komplexer und dynamischer Systeme zu erzielen, hat zur Entwicklung von Co-Simulations Testumgebungen geführt, die viele domänenspezifische Simulatoren miteinander verbinden, um die Forschung an Interaktionen zwischen Energie- und Rechen-systemen voranzutreiben.

Diese Arbeit stellt eine Erweiterung des kürzlich entwickelten Vessim Frameworks, das als eine solche Testumgebung dient, vor, um die Integration und Verhaltensweisen verschiedener Batteriemodelle in Simulations-Szenarien von Rechenzentren zu ermöglichen. Um die Nützlichkeit der Verwendung spezifischer Modelle für gängige Anwendungsfälle zu ermitteln, führt diese Arbeit eine Analyse durch, bei der vier separate Modelle für Batterie-Pakete mit Lithium-Ionen-Zellen in das Framework eingebettet werden. Darunter sind sowohl einfache Modelle, die keine Verluste oder Grenzwerte berücksichtigen, als auch komplizierte Modelle, die elektrochemische Reaktionen und Ineffizienzen auf Schaltungsebene betrachten.

Die Ergebnisse der Analyse zeigen, dass die Verwendung von physikbasierten Batteriezellenmodellen wie SPMs zu langen Ausführungszeiten von Co-Simulationsszenarien führen. Das lineare C-L-C Modell, das Ineffizienzen und Leistungsbegrenzungen in Abhängigkeit vom Zustand der Batterie beachtet, kann das Verhalten dieser Modelle in allen Kurzzeitszenarien mit einer wesentlich höheren Simulationsgeschwindigkeit recht genau wiedergeben, was es zu einer guten Wahl für die meisten Experimentierzwecke macht. Einfache, verlustfreie Modelle, die den Stromfluss nicht einschränken, sind im Allgemeinen nicht in der Lage, die Leistung komplexer Modelle wiederzugeben und haben auch keine weiteren Laufzeitvorteile.

Abstract

In times of growing demand for computing resources, the call for enhanced energy sustainability leads to changes in the infrastructure of data centers' power systems. Many providers are now opting for the operation of their compute nodes in microgrid environments, as this approach offers better control and flexibility, allowing for more efficient regulation of energy use and adaptation to the varying production of renewable power sources. The necessity to find advancements in the management of such complex and dynamic systems has led to the development of co-simulation testbeds, connecting domain-specific simulators to facilitate research on interactions between power and computing systems.

This thesis proposes an extension to the recently developed Vessim framework, which serves as such a testbed, to enable the integration and study of various battery models in data center scenarios. To determine the usefulness of utilizing specific models for common use cases, this work conducts an analysis, implementing four separate models for battery packs with lithium-ion cells in the framework, including both simple models that do not consider any losses or limits, and intricate models, accounting for electrochemical reactions and circuit-level dynamics.

The results of the analysis indicate that the utilization of physics-based battery cell models such as SPMs come at the cost of large execution times in co-simulation scenarios. The linear C-L-C model, which acknowledges inefficiencies and power limitations depending on the battery's state, can accurately reflect their behavior in all short-term experiments with a massively faster simulation speed, making it a good choice for most experimentation purposes. Simple, lossless models that do not limit the power flow generally lack the ability to produce decent representations of the complex models' performance, while not yielding further benefits in terms of runtime.

Contents

1	Introduction	2
1.1	Outline	3
2	Background	5
2.1	Microgrids	5
2.2	Co-Simulation	6
2.2.1	The Mosaik Framework	7
2.2.2	Vessim Architecture	8
2.3	Battery Management Systems	10
2.4	Lithium-Ion Battery Models	10
2.4.1	C/L/C Model	12
2.4.2	Single Particle Models	14
3	Related Work	16
3.1	Microgrid/Energy System Modeling	16
3.2	System Testbeds	17
3.3	Battery Simulation	17
4	Storage Simulator Integration	19
4.1	Extended Co-Simulation Architecture	19
4.2	Energy Storage Interface	20
4.3	Energy Management Policies	22
5	Battery Model Analysis	23
5.1	Battery Models for Co-Simulation	23
5.1.1	SimpleBattery	23
5.1.2	CLCBattery	24
5.1.3	PybammBattery	26
5.1.4	LiionBatteryPack	27
5.2	Experimental Results	29
5.2.1	Constant Power (Dis-)Charging	30
5.2.2	Data Center Scenario	31
5.2.3	Runtime Analysis	33
5.3	Discussion	34
6	Conclusion	37

Chapter 1

Introduction

As the adoption of modern data-intensive technologies such as artificial intelligence continues to accelerate across industries, the need for computational power is rising and has been for some time [1]. Consequently, today’s hyperscale data centers are steadily increasing their electricity use, with 1% of the global energy consumption being attributed to server farms [2]. With an introduction of carbon pricing mechanisms on the horizon [3, 4], this power demand and the subsequent carbon footprint will become an issue to the operators if their infrastructure is solely powered by the public grid, as fossil fuels produce a big portion of the energy in today’s world [5].

Many cloud providers like Microsoft ¹ and Google ² have already stated ambitious plans to reduce their environmental impact, which requires them to utilize the promising alternative of distributed renewable energy resources to meet their power demand [6]. In this context, microgrids provide a fitting framework for defining the operation of distributed generation and consumption [7], and recent work has outlined the advantages of microgrids over traditional power systems in terms of control, flexibility, economic viability, environmental friendliness, and adaptability [8]. Because of the rather volatile nature of renewable energy sources like solar and wind, the energy output of a localized microgrid will fluctuate over time, which makes the use of energy storage systems such as batteries necessary to deal with upcoming imbalances between demand and supply side [9].

In this complex environment, the task of studying dynamic behaviors like load-, energy-, and battery management becomes increasingly difficult [10], and the process of researching and developing system configurations, taking these things into account remains challenging due to the limited availability of suitable test environments. Recent work has therefore proposed a tool called Vessim [11] to serve as a virtual testbed for carbon-aware systems, where applications have visibility and control over a simulated microgrid. Vessim utilizes co-simulation, which offers a platform to study coupled het-

¹<https://blogs.microsoft.com/blog/2020/01/16/microsoft-will-be-carbon-negative-by-2030/>

²<https://www.gstatic.com/gumdrop/sustainability/google-2019-environmental-report.pdf>

erogeneous systems like microgrids by composing simulations of its different parts [12]. This is sensible because at its core, a microgrid comprises an interplay of various components like renewable power sources, energy storage systems, power electronics, and intelligent control algorithms [13].

Thus far, however, the opportunities of modeling energy storage systems in such testbeds are severely limited. Either only way too simple battery models with nothing more than an adjustable energy level are supported, or one has to work with too complex and slow simulations of a complete underlying microgrid power network, to study accurate physics-based storage models that consider parameters like battery aging and thermal factors. No work has been done, specifying the interfaces that are necessary to integrate battery models of different complexity into a microgrid co-simulation, and analyzing, how these various models compare.

1.1 Outline

The objective of this thesis is to investigate, how different battery models can be integrated into a microgrid co-simulation, which can simulate the power system infrastructure of a data center. These models can range from simple, lossless batteries with the sole purpose of transferring usable energy over the duration of a simulation, to complex models of entire battery packs that take the internal physical processes of batteries as well as the energy losses of the battery packs circuit into account.

The work starts by providing relevant background information on topics such as Microgrids, Co-Simulation, and Battery Modeling as well as used frameworks for experimentation like Mosaik [14] or Vessim [11] in Chapter 2. Afterward, Chapter 3 presents an overview of related research in these areas.

The goals of this thesis are achieved by extending the already existing co-simulation architecture of the mentioned Vessim framework, and add more context to specific parts of the design. For this purpose, mainly the storage simulation unit is refined and explained in detail so that the described objectives can be met, and the architecture can be used as a co-simulation testbed for data centers. This system design is described in detail in Chapter 4.

This work then describes the implementation of four different mathematical models of lithium-ion battery packs with battery management systems, using the proposed interface for energy storage units:

1. The simplest model (**SimpleBattery**) simulates a basic, lossless battery that has a fixed capacity, and does not consider any inefficiencies or physical properties of a battery pack.
2. The **CLCBattery** implements the C-L-C model as described by Kazhamiaka et al. [15] that is still a simple linear mathematical model for single lithium-ion battery cells, but considers properties such as charging and discharging inefficiencies and power limits.

3. The **PybammBattery** uses the PyBaMM framework [16] to simulate individual lithium-ion cells that behave according to the Single Particle Model, which takes the electrochemical processes inside a battery into account.
4. Finally, the **LiionBatteryPack** models the behavior of a full lithium-ion battery pack, where the cells are again emulated using PyBaMM’s Single Particle Model, and the battery pack’s circuit is solved using methods of the Liionpack framework [17].

The enhanced co-simulation design of Vessim [11] is used to analyze, how these different models behave in different scenarios within a simulated microgrid, which can represent a data center’s power system. The description of the models, the experimental results, and their discussion can be found in Chapter 5, before Chapter 6 concludes this thesis by summarizing the key findings of this work.

Chapter 2

Background

This chapter provides a comprehensive overview of relevant concepts that are essential for this work. Section 2.1 introduces the principles of microgrid infrastructures, while Section 2.2 focuses on co-simulation and with that describes different approaches to model complex systems. Finally, after a brief description of battery management systems in Section 2.3, Section 2.4 presents models for lithium-ion batteries, which are used for evaluating the approach.

2.1 Microgrids

Economic, technological, and environmental factors are driving changes in how electricity is generated and transmitted. The dominance of large centralized power plants is diminishing and giving way to smaller, more decentralized models, partly due to the decreasing advantages of traditional economies of scale [18]. The concept of a microgrid was first presented by Lasseter [7] as early as 2002, describing a paradigm to define the operation of distributed generation. The core idea is to perceive such generators and associated loads as a system of its own to allow for local control, thus providing a higher local reliability and efficiency.

The CIGRÉ Working Group C6.22 *Microgrid Evolution Roadmap* defines the following components as the foundational elements of a microgrid [19]:

Generators encompass various energy sources suitable for microgrids, including fossil or biomass-fired small-scale combined heat and power, photovoltaic modules, wind turbines, etc.

Storage Devices include diverse technologies such as electrical, electrochemical, mechanical, and heat storage.

Controlled loads refer to all kinds of electrical loads within a microgrid that can be managed or controlled to optimize energy usage and system performance.

Additionally, a microgrid can operate in two different modes: grid-connected and

islanded [18, 19, 20]. In the grid-connected mode, the microgrid functions as a part of the larger utility grid, while drawing power from the grid as needed, enabling it to benefit from the main grids stability and reliability. During certain phases like grid outages, the microgrid is capable of a seamless transition into islanded mode where it disconnects from the grid and operates autonomously, relying solely on its own distributed energy resources to meet local energy demands. Effective control and coordination mechanisms are consequently essential for smooth transitions between these two modes, to optimize energy management and ensure reliable operation under varying conditions.

2.2 Co-Simulation

The purpose of simulations is to recreate a real-world system or scenario in a controlled environment, and it is a powerful tool in that regard. They eliminate the risks and costs associated with doing certain experimentation on a real system or hardware while testing relevant hypotheses, and, as a result, allow insights into complex scenarios by predicting real-world outcomes. In a world of ever-increasing system complexity, co-simulation presents a way to leverage from established tools for the simulation of subsystems, by allowing the concurrent execution of different simulators [21]. In that way, more complex systems can be built by simply defining the interactions between them.

Co-simulation encompasses the theoretical framework as well as practical methodologies, involving simulation of interconnected systems through the integration of multiple simulators. In this approach, each simulator serves as a self-contained entity, analogous to a black box, possessing the capability to emulate specific behaviors, consume inputs, and generate outputs [12]. Therefore, the concept of co-simulation does not attempt to fit an entire system into a single monolithic simulation environment, but instead separates diverse subsystems, which have their distinct computational models or, in some cases, different physical laws.

In a co-simulated scenario, each individual component or subsystem has its simulation unit that produces a behavior trace (or simulation) when given inputs, and, if it is not a real-world entity, has the following components: A Model of the system under study that is a dynamical system with a state and a notion of evolution rules, and a Solver/Simulator that works as an algorithm computing or approximating the behavior trace of a model [12, 21]. When these simulation units are coupled using their inputs and outputs, they produce a combined behavior trace of the entire modeled system, which is called co-simulation.

Because the simulation units are treated as independent black-boxes, to run a co-simulation, a separate orchestrator is needed, next to the co-simulation scenario, to handle the coupling at the behavioral level. The orchestrator is responsible for the initialization of the simulation units, the connection of their inputs and outputs, and the progression of simulated time in the individual components. Gomes et al. [12] defines three different approaches for the operation of orchestrators:

In **Discrete-Event** based co-simulation, the scenario consists of multiple discrete-event simulation units, which means that they communicate with the environment using timestamped events. Their internal state can change despite the simulation time being stopped (reactivity), and events can trigger other events instantaneously (transiency). The orchestrator hereby handles the mapping between output events of one unit to external events in another unit.

In **Continuous-Time** based co-simulation, the state of each continuous-time simulation unit evolves constantly. The orchestrator has to set a communication step-size, defining the times when the units exchange their input/output values because the simulation units each have independent micro step-sizes.

Finally, in **Hybrid** co-simulation, the characteristics of both kinds of simulation units are combined. The orchestrator can be either discrete-event based or continuous-time based, whereas the behavior of the simulation units have to be wrapped to become discrete-event or continuous-time based units.

Within a co-simulation environment, each simulation unit is operated as an independent module using its individual simulation algorithm. By selecting and implementing appropriate models for each subsystem, this modularity ensures that the models are more specialized and tailored to the characteristics of its modeled subsystem. With the described black-box approach, co-simulation treats individual components as interchangeable, which means that as long as the inputs and outputs of a component adhere to a predefined interface, they can be swapped with alternative components without making extensive modifications to the general system architecture necessary. Therefore, co-simulation allows for more flexibility compared to a monolithic approach.

Following this, the next sections will present relevant frameworks for this work using co-simulation, where Section 2.2.1 explains the Mosaik framework [14], and Section 2.2.2 introduces Vessim [11], which is used for evaluating the approaches of this thesis and builds on top of Mosaik.

2.2.1 The Mosaik Framework

Mosaik [14] was designed to be used as a co-simulation framework to study scenarios in the area of the smart grid [22], and can, for instance, serve as testbeds for different control strategies in said smart grid. Nowadays, however, it can easily be used as a flexible co-simulator for various environments by combining different simulation units to create large-scale scenarios.

At its core, Mosaik is responsible for synchronizing processes that each run a simulation, and to manage data-flows between them. To achieve this, the framework encompasses four main components ¹:

Mosaik API: Establishes a common protocol for communication between the

¹These main components are listed in the Mosaik documentation under the following link: <https://mosaik.readthedocs.io/en/3.3.0/overview.html>

Mosaik-core and the coupled simulators. While the low-level API is based on network sockets for the passing of JSON messages, there are also high-level APIs available for various programming languages, which provide an extra abstraction layer by handling the networking.

Scenario API: Python API for creating simulation scenarios, which makes it possible to define data flows by connecting simulation units, and to execute these scenarios by starting the different simulation units.

Simulation Manager: Handles external simulator processes and communicates with them. They are internally represented by proxy objects so that it makes no difference to other Mosaik components if the simulators are in-process with the Mosaik-core, started by Mosaik itself, or if Mosaik simply connected to a running simulator instance.

Mosaik Scheduler: The discrete-event orchestrator of the co-simulation. It is responsible for stepping the simulators through time while keeping them synchronized, and for the data passing of inputs and outputs between the simulation units. Mosaik allows these simulation units to be continuous-time based or hybrid by wrapping them in discrete-event simulation units.

2.2.2 Vessim Architecture

With the goal of accelerating research in the area of sustainable computing, Wiesner et al. recently presented the Vessim Framework [11], which evolved in the last year to provide a co-simulation test environment for the development of carbon-aware applications and systems that interact with their power system. It allows users to define individual microgrids to represent the power systems of computing infrastructure like data centers, utilizing the described Mosaik framework under the hood. Each microgrid consists of multiple simulation units responsible for modeling generators, consumers, and energy storage as well as user-defined controllers, which can provide applications visibility and control over the power system.

The co-simulation architecture of a single microgrid, which is depicted in Figure 2.1, is defined using the Mosaik Scenario API, and the individual simulation units implement the high-level Python API of Mosaik. A Vessim microgrid consists of four distinct subsystems that are stepped in a specific order, whereas the Mosaik Scheduler handles data passing and synchronization.

1. **Actors:** The actors model the energy consumers and producers of the microgrid, and are characterized by their individual power trace $p^{<\text{actor}>}$. Every actor has its separate Mosaik simulation unit, which forwards the actor's power production ($p_t^{<\text{actor}>} > 0$) or power consumption ($p_t^{<\text{actor}>} < 0$) at every simulation step t . The actor models are highly variable as they can be everything from simulated nodes, which only read out historical traces of, for instance, solar irradiance or computation load, to nodes that monitor full on server farms, running hundreds of virtual machines where the power consumption is measured using power meters.

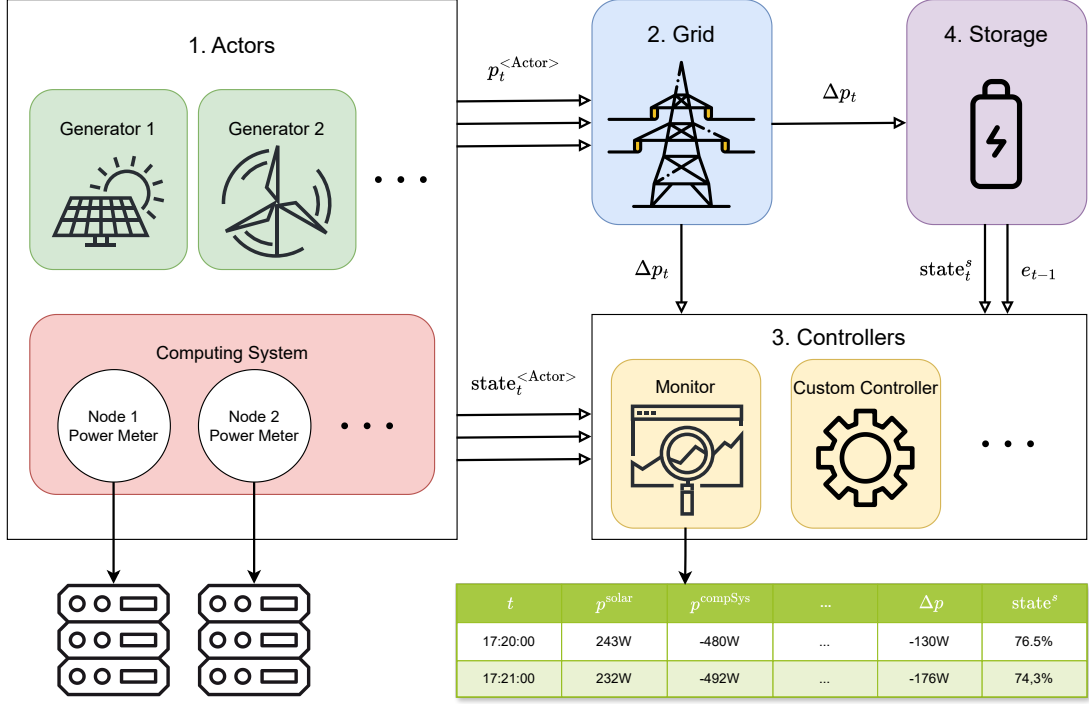


Figure 2.1: Microgrid co-simulation architecture as described by Wiesner et al. [11]. Colored components mark Mosaik simulation units.

Due to this high diversity, the actors also have the option to provide controllers with additional information to allow for better decision-making by transmitting their state ($\text{state}_t^{\text{actor}}$).

2. **Grid:** The Grid simulator is responsible to model the power flow inside the microgrid's power network. At every simulation step t , it gets the power trace of the actors as input, and produces a power delta Δp_t that can be stored to or has to be discharged from the energy storage. To accurately model this power flow, frameworks like PyPSA [23] can be used, but for many cases, a simple aggregation $\Delta p_t = \sum p_t^{\text{actor}}$ is enough.
3. **Controllers:** Users can define custom controllers that can monitor and configure the power and computing systems during the simulation. Controllers receive the entire state of the microgrid's power system at every simulation step t . Next to the state of all the actors ($\text{state}_t^{\text{actor}}$) and the state of the energy storage (state_t^s), this also includes the current power delta (Δp_t) as well as the excess energy or energy deficit that the power system had since the last time step (e_{t-1}). With all this information, which can also be extended by other signals like production and load forecasts, the controllers have the flexibility, next to monitoring, to also reduce consumption by throttling the computing system or tweak the energy storage configuration.

4. **Storage:** The storage simulator manages the charging and discharging of the microgrid’s energy storage (if present), and the exchanging of energy with the public utility grid. With that, it essentially models the progression of time between simulation time steps, while the other subsystems are effectively executed simultaneously at the time steps. It gets the computed (Δp_t), and provides information regarding the new state of the storage (state_{t+1}^s), and the amount of energy that has been drawn from/ fed into the public grid between the time steps to the controllers (e_t). Because the storage simulator is stepped after the controllers, the controllers always receive the information from the previous time step.

This thesis aims to extend upon the Vessim approach by further specifying the storage subsystem of the discussed simulated microgrid. This refined architecture will then be used together with simple and complex battery models to show that the described environment can be used as a co-simulation testbed for data centers that allows for the use of variable energy storage modeling approaches.

2.3 Battery Management Systems

A Battery Management System (BMS) is a key part of an energy storage system because, in the context of a microgrid, it is responsible to monitor and control the energy storage functions. The central goal of using a BMS is to operate the corresponding battery in an efficient, safe, and reliable manner [24, 25].

To avoid unsafe conditions, a BMS can, for instance, prevent the battery from deep discharging, overcharging in an already full state, or exceeding limits like voltage cutoffs, charging or discharging currents typically defined by a charging rate (C-rate), or internal battery temperature [24]. All that could cause damage to the battery’s structure, and thus, the BMS serves as a safety net in that regard.

For various systems, it is also necessary to estimate a battery’s state, which can be achieved using a BMS. Some of the most important metrics are the State-of-Charge (SoC), and the State-of-Health (SoH). The SoC describes the available energy still present in the storage system and is typically presented as a percentage of its total energy capacity, whereas the SoH is defined as the maximum possible battery charge compared to the battery’s capacity, and informs on battery aging. Note that both of these metrics cannot be measured directly, but have to be estimated in a real-world scenario [24].

2.4 Lithium-Ion Battery Models

The main use cases of lithium-ion batteries (or li-ion batteries) are electric and hybrid vehicles as well as portable electronic devices, due to properties such as their energy density, long cycle life, and low self-discharge rate [26, 27]. The main components of those batteries are the positive electrode (Cathode), the negative electrode (Anode), an

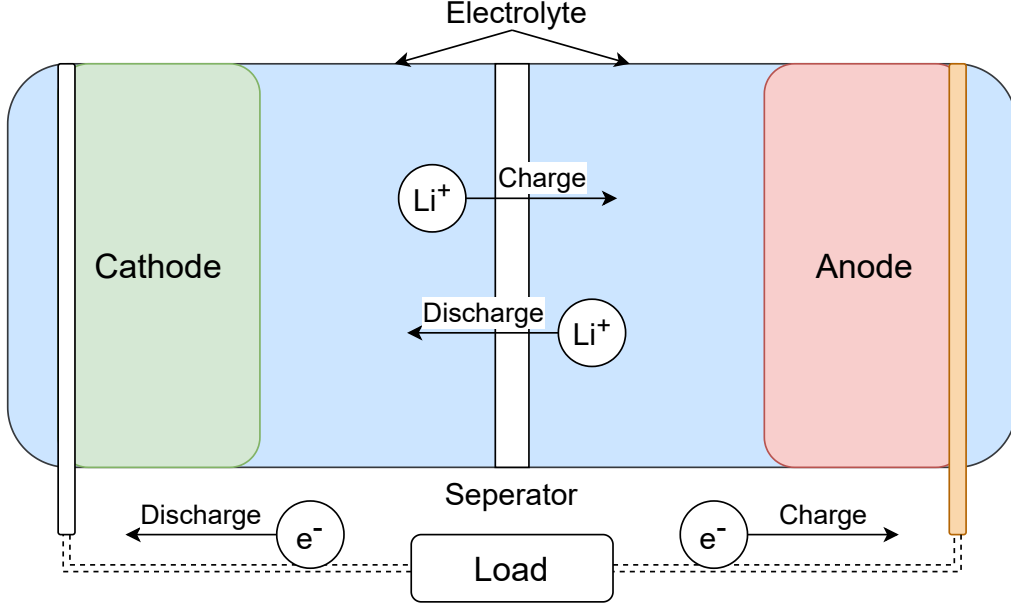


Figure 2.2: Simplified representation of the internal structure of a lithium-ion battery.

electrolyte fluid, and a separator, preventing physical contact between the electrodes. A simplified structure of a lithium-ion battery is depicted in Figure 2.2.

When an external voltage is applied to the battery during charging, it forces lithium ions from the cathode, which contains a lithium metal oxide, to move through the electrolyte to the anode, where it is inserted into graphite layers [27]. The electrons released during the electrochemical reaction cannot pass through the separator, and instead flow through the external circuit to balance the charge after being collected by a metal sheet.

During discharging, the opposite process occurs, as electrons move through the external circuit to the cathode, inducing a current to be used by the load, while the lithium-ions pass the electrolyte to become once again part of the lithium-metal-oxide [27]. Because the difference in electrical charge decreases over time, the terminal voltage of the battery falls as it empties.

The next subsections present a quick overview of relevant models of lithium-ion batteries that are applied in this thesis, whereas battery modeling describes the process of creating mathematical representations to predict the transformation of the battery state under varying conditions such as charging, discharging and resting. Section 2.4.1 discusses the C/L/C model as described by Kazhamiaka et al. [15], and Section 2.4.2 presents single particle models for lithium-ion batteries.

2.4.1 C/L/C Model

The C/L/C model is a tractable implementation of the Power-based Integrated (PI) Model as proposed by Kazhamiaka et al. in 2018 [28], which tries to estimate the energy content of a lithium-ion battery cell. In the PI model, the battery consists of a Battery Management System (BMS) as well as lithium-ion cells, and the change of the battery system's energy level is modeled while the storage is being charged or discharged at varying power over time.

In every time-step t , the PI model gets the applied power $p(t)$, the energy content of the previous time-step $b(t-1)$, as well as the duration of the time-step d as inputs to compute the battery's energy $b(t)$. This is achieved by adding a specific energy delta (as shown in equation 2.1), which depends on all these inputs ².

$$b(t) = b(t-1) + \Delta_E(t) \quad (2.1)$$

The mentioned energy delta of a step t is computed by multiplying the applied power $p(t)$ ³ times the duration d and a (dis-)charging penalty, marking the power loss due to inefficiencies (see equation 2.2). The charging inefficiency $\eta_c(I, V)$ describes a function representing the fraction of power stored in the battery while it is charged with a certain power p , while $\eta_d(I, V)$ (discharging inefficiency) stands for the function representing the fraction of power that has to be discharged from the battery to obtain the power p . Subsequently, $0 \leq \eta_c(I, V) \leq 1$ and $\eta_d(I, V) \geq 1$ hold true. Note that the charging and discharging inefficiencies are dependent on the battery's current $I(t)$, and the battery's voltage $V(t)$.

$$\Delta_E(t) = \begin{cases} \eta_c(I(t), V(t)) \cdot p(t) \cdot d & | p(t) \geq 0 \\ \eta_d(I(t), V(t)) \cdot p(t) \cdot d & | p(t) < 0 \end{cases} \quad (2.2)$$

The PI model has an internal representation of a battery's terminal voltage V , which is computed at every time-step t . The voltage of the battery is described by a function M , which relates the battery's energy content $b(t)$ and the applied current $I(t)$ to its voltage, as presented in equation 2.3. Typically, the terminal voltage rises when the battery is charged, and drops when it is discharged relative to its open-circuit voltage in idle state.

$$V(t) = M(b(t), I(t)) \quad (2.3)$$

The relation between the battery's applied power p , its applied current I , and its terminal voltage V at time-step t yields the internal representation of the current function over time:

²The equations for the PI model in this chapter are taken from the paper by Kazhamiaka et al. [28].

³ $p(t) > 0$ indicates charging and $p(t) < 0$ indicates discharging.

$$I(t) = \frac{p(t)}{V(t)} \quad (2.4)$$

The BMS attached to the batteries limits the charging and discharging currents to prevent overheating and degradation of components, which is done by adding the constraint 2.5 to the model. The parameters α_d (discharging current limit) and α_c (charging current limit) are constants that can normally be found on a battery model's specification sheet.

$$\alpha_d \leq I(t) \leq \alpha_c \quad (2.5)$$

Lastly, the PI model has another constraint regarding the battery's applied current. Because a battery's voltage should not exceed certain thresholds during charging or discharging to prevent damage, and that terminal voltage rises when charged with a higher current (or falls when discharged at a higher current), the energy that can be fed to or taken from the battery varies depending on the (dis-)charging current. Therefore, an energy content range is specified, where the BMS prevents the batteries from being discharged further than a lower energy content limit $a_1(I)$, or being charged further than an upper energy content limit $a_2(I)$. These limits mark the point where the voltage reaches its allowed minimum or maximum while being (dis-)charged. The following constraint is consequently added to the PI model:

$$a_1(I(t)) \leq b(t) \leq a_2(I(t)) \quad (2.6)$$

Given this base model, Kazhamiaka et al. describe a set of tractable implementations in their 2019 paper [15]. The main issue is that the three principal components (the M function for computation of internal voltage, the energy content limit functions a_1 and a_2 , and the inefficiency functions η_c and η_d) have no explicit mathematical representation or are non-linear, which makes mathematical optimization difficult.

In the tractable models, these components are thus approximated with polynomials. The C/L/C model, for instance, uses the following approximations to retrieve a mathematical model for lithium-ion batteries:

- **Constant approximation** for the battery's terminal voltage. This could be, for instance, the nominal voltage which can be found in the specifications ($V(t) = V_{nom}$).
- **Linear approximation** for the energy content limits depending on the current: $a_1(I(t)) = u_1 I(t) + v_1$ and $a_2(I(t)) = u_2 I(t) + v_2$
- **Constant approximation** for the battery's charging and discharging inefficiency: $\eta_d(I(t), V(t)) = \bar{\eta}_d$ and $\eta_c(I(t), V(t)) = \bar{\eta}_c$

All in all, the C/L/C model has linear complexity while offering a decent estimation of a battery's energy level over time, but it can be seen that the discussed electrochemical

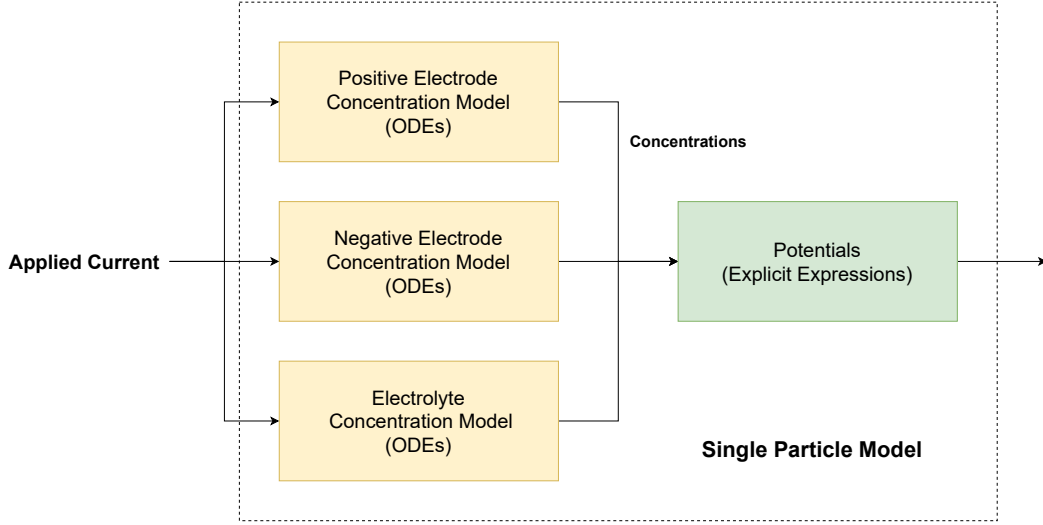


Figure 2.3: Representation of a Single Particle Model with its submodels. The figure is adapted from Planella et al. [32] with modifications.

reactions in the battery are at no point considered. This model is, for instance, used by the Carbon Explorer framework [29] to represent the present energy storage of data centers.

2.4.2 Single Particle Models

The Doyle-Fuller-Newman (DFN) Model, which was developed in 1993 by Doyle et al. [30], is the de facto standard mathematical model for describing the electrochemical processes inside lithium-ion batteries. It is, however, too computationally complex for many applications as it consists of coupled non-linear partial differential equations. There is often no need to spend that many computational resources on such sophisticated models when, for instance, battery packs are studied [31, 32], which motivates the deployment of simpler physics-based models for analyzing larger energy storage systems.

Single particle models (SPMs) are considered to be the least complex electrochemical models for lithium-ion batteries [31, 33], and are based on the assumption of the particles in the electrodes behaving almost the same when the battery is used at low to moderate charging rates. Thus, instead of looking at all these tiny particles separately, the model represents each electrode by a single particle to simplify the calculations, as the problem of the movement of lithium-ions in the electrodes has to be solved only for two distinct particles [32]. Because the total electric charge entering a system must be equal to the total charge leaving the system, the current that is applied to the battery can be related to the rate in which lithium-ions are removed or inserted into the electrode particles, which means that the behavior of the entire electrode and its concentration can be calculated when the applied current is known.

Some Single Particle Models also consider the behavior of the electrolyte [34] in addition to the representation of the electrodes, and in similar fashion, the concentration of lithium-ions in the electrolyte can be determined. Eventually, the three described submodels can be solved using ordinary differential equations (ODEs), which are easier to handle than the coupled non-linear partial differential equations of the DFN model [32]. As presented in Figure 2.3, these determined concentrations can be used to compute battery potentials like the terminal voltage by evaluating explicit mathematical expressions with again limited computational complexity.

In conclusion, SPMs are a group of computationally inexpensive electrochemical battery models that consider physical battery traits. There also exist some open-source frameworks like, for instance, PyBaMM [16] and Liionpack [17] that can be used for the simulation of batteries and battery packs that already include solvers for a Single Particle Model with and without considering the electrolyte. These will later be used for the model analysis in Chapter 5, which is why the exact mathematical representations of the discussed principles are not mentioned at this point, as they have no relevance for this work.

Chapter 3

Related Work

This chapter presents and reviews related research in the areas of this thesis. Section 3.1 discusses different papers on modeling and simulating energy systems of a microgrid. Section 3.2 presents work regarding application and system configuration testbeds, and Section 3.3 focuses on current research for battery simulation.

3.1 Microgrid/Energy System Modeling

Li et al. [5] proposed a comprehensive energy management framework that facilitates a sustainable edge computing paradigm with distributed renewable energy sources. Their approach aims to integrate microgrid and edge computing technologies to increase energy resource efficiency by enabling collaboration between the systems. It is evaluated by the development of a hardware prototype system with a solar-wind hybrid power system, batteries, and two servers, acting as an edge computing system. However, the authors did not address software-based modelling and simulation approaches for such a system of systems, and instead presented a design idea for future resource management in an edge-computing environment.

Alzahrani et al. [13] focused on presenting various models of energy system parts, and simulated a small-scale microgrid, while treating all the different components as autonomous systems. The work further describes system modelling, utilizing time series neural networks to generate solar and wind power data as well as load profiles and storage state-of-charge estimations during the simulation. Despite their discussion of battery models, the simulations used for the evaluation did not include a battery system, and instead treated the system as purely grid-connected.

In contrast to the above papers, Kounev et al. [35] investigated the use of co-simulation to evaluate the performance of a microgrid infrastructure utilizing embedded power controllers. In particular, the work distinguishes between two different networks within the microgrid: The power distribution network and the data communication network. Additionally, the authors proposed a new co-simulation scheduler to catch events

from both network simulators, and synchronize the simulators, but the whole power network still consists of a single simulator, making it difficult to examine various energy storage models effectively.

In 2017, Abrishambaf et al. [36] implemented a framework supporting real-time simulation of a microgrid. The implementation allows for two different methods of control (centralized and distributed) that the operator can choose, and their case study consists of 33 buses, 220 consumers, and 68 distributed generators. Their approach, however, heavily relies on hardware to simulate the individual components of the microgrid, and their case study model again lacks any kind of energy storage and instead assumes a connection to a larger distribution network.

3.2 System Testbeds

The aforementioned Vessim framework, proposed by Wiesner et al. [11], aims to serve as a co-simulation testbed for carbon-aware applications and systems. Vessim establishes standardized interfaces, empowering users to flexibly configure their (partially) simulated power systems. It enables seamless integration with actual software and hardware, offering a range of utility functions such as replaying power production forecasts or accessing live data through queries. Under the hood, power systems are simulated by the connection of domain-specific simulators for renewable power generation, distributed power consumption, and energy storage, utilizing the Mosaik co-simulation framework [14]. Even though, their approach facilitates research into the diverse facets concerning the interaction between power and computing systems, so far, their efforts regarding battery modeling have not exceeded way too simple battery models.

Acun et al. [29] proposed the Carbon Explorer as a framework for reducing data center’s carbon footprint, targeting the operational as well as the embodied carbon emissions of the data center ecosystem in a holistic approach. In this context, operational carbon refers to the emissions related to the computing system’s energy consumption, and embodied carbon encompasses emissions associated with production, utilization, and disposal of relevant IT infrastructure. The framework also contains an energy simulator, accounting for renewable- and battery deployment, and carbon-aware scheduling to calculate the data center’s operational carbon emissions. Carbon explorer utilizes a modular C/L/C battery model that includes several characteristics of lithium-ion batteries to simulate their energy storage, but more sophisticated battery modeling is still not considered.

3.3 Battery Simulation

PyBaMM [16] (Python Battery Mathematical Modeling) is an open-source Python library for modeling and simulation of battery systems. The framework offers a wide range of mathematical models to describe the electrochemical processes inside batteries, including the comprehensive physics-based DFN model and the more efficient SPM model for

lithium-ion batteries. In addition to that, PyBaMM allows users to define complex battery geometries and generates computational meshes automatically, and facilitates the parameterization of battery models by providing tools to calibrate and validate against experimental data, which enables accurate simulation of battery behavior under various operating conditions.

The liionpack framework [17] builds on top of PyBaMM, allowing users to scale up their simulations from a single battery cell to a battery pack of multiple lithium-ion batteries. With modeling of interconnections with defined resistances, a group of batteries can be connected in series and parallel to define a pack architecture, which is then solved by processing the global circuit before the electrochemical problem for each battery is considered. Although both of these libraries allow for state-of-the-art battery modeling, they do not provide an interface to allow coupling with other simulators for a power system simulation.

Chapter 4

Storage Simulator Integration

This chapter presents the architecture used to enable the integration of energy storage models into a co-simulation. Section 4.1 elaborates on the existing Vessim architecture for energy storage modeling, and extends it. Afterward, Section 4.2 describes the interface this approach uses for the implementation of battery pack models at multiple levels of complexity, whereas Section 4.3 discusses, how policies are defined, which are used to balance the energy within a microgrid simulation at every time-step, and are necessary to perform the required energy management in the co-simulator.

4.1 Extended Co-Simulation Architecture

The structure of a co-simulated microgrid, according to Wiesner et al. [11], was already presented in Section 2.2.2. It is now being refined to include an energy storage interface, energy management policies, and new connections between the storage simulation unit and the controller simulators. Figure 4.1 depicts the resulting architecture.

The defined **Storage** and **MicrogridPolicy** objects, detailed in the following sections, are included in the Vessim storage simulator, which is the last simulator to step, modeling the progression of time between time-steps. As mentioned, at every time-step t , the storage simulation unit receives Δp_t through the Mosaik Scheduler (see Section 2.2.1). This Δp_t marks the power delta of the energy producers and consumers, as obtained by the grid simulation unit utilizing a power flow analysis, and, along with the duration of the time-step d , is passed to the **MicrogridPolicy** object. The value of d is known because the storage simulation unit is continuous-time based while implementing the Mosaik API, and therefore includes a step-size. Using these parameters, the policy is applied as explained in Section 4.3, utilizing the energy storage interface defined in Section 4.2.

The return value of that function call indicates the energy e_t taken from or fed into the utility grid during the last time-step t . This value is passed to the controllers using the Mosaik Scheduler, together with the updated state of the storage $state_{t+1}^s$ (defined

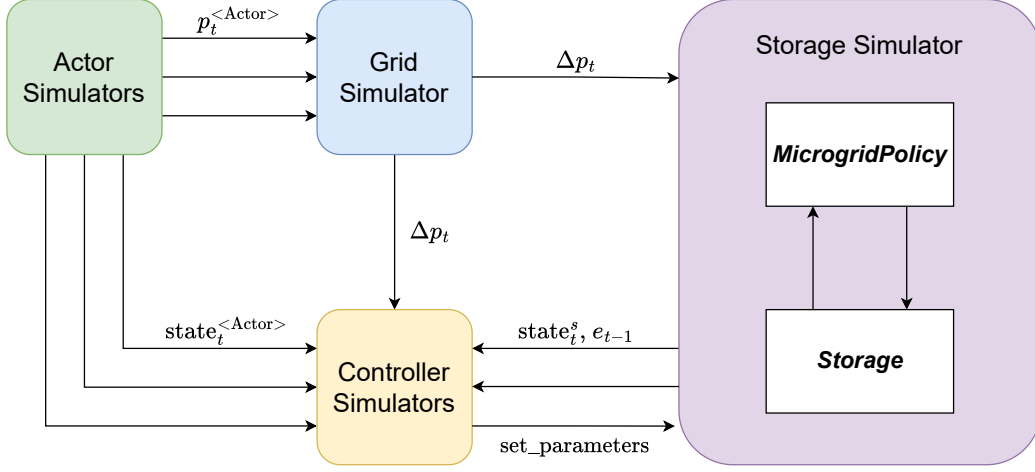


Figure 4.1: Refined Co-Simulation structure with the proposed energy storage interface.

in Section 4.2). That way, the architecture provides the controllers, and thereby also carbon-aware applications, *visibility* over the power system, which is one of the key ideas behind the Vessim framework. Because the storage simulation unit is stepped after the controllers, the message passing between the simulation units is cyclic, meaning that the controllers receive the energy delta of the previous time-step $t - 1$, and the storage state at the beginning of the time-step t .

In addition to this provision of *visibility*, another requirement of the Vessim architecture is to enable *control* over the simulated microgrid. In terms of the energy management and the energy storage, the topics that this thesis focuses on, *control* means that the controllers can tweak the configuration of the energy storage system during simulation runtime, as a paper by Wiesner et al. [37] already describes how applications can interact with the controllers via an API through Software-in-the-loop simulation. This is achieved again using the message passing of the Mosaik Scheduler, as the controllers can send a collection of key-value pairs to the storage simulation unit, where they are handled so that relevant functions for the updating of parameters are called on the **MicrogridPolicy** or **Storage** objects. Thus, the responsibility for defining the parameters that can be updated during runtime lies with the developer of the models and policies.

4.2 Energy Storage Interface

One of the main goals of this work is to describe a common interface for energy storage models to be used inside a microgrid co-simulation. This interface should be adaptable enough so that the complexity range of these models is not too limited, but has to be simple enough to allow a clear co-simulation architecture that still separates power consumption and power generation from the process of storing energy.

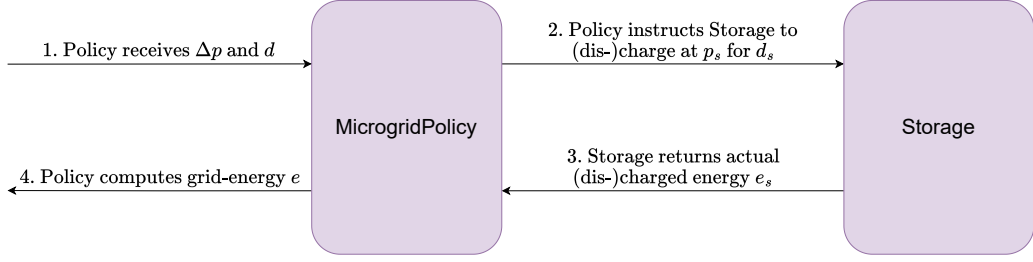


Figure 4.2: Interactions between the **MicrogridPolicy** and the **Storage**.

Due to the underlying Vessim architecture [11], which conducts power flow analysis, the main input to the battery models is power instead of current, which would be the common input to complex models like the SPM (see Section 2.4.2). Additionally, a management system is modeled as part of the battery, similarly to what is described by Kazhamiaka et al. [28] in their paper discussing the PI-model as described in Section 2.4.1. The reason behind that is to not allow simulated batteries to be used in ways that would be prevented by a BMS (see Section 2.3) like over- or undercharging, and to again provide the necessary *visibility* over the storage.

In the proposed design, the **Storage** provides the abstract interface for a battery system, which allows for charging and discharging operations, and contains the monitoring and control functionality of a BMS. The primary function of the interface is to handle the energy flow into and from the battery, which is facilitated through an update method, receiving the power to be (dis-)charged p_s as well as the (dis-)charging duration d_s . The **Storage** adjusts its internal state based on these parameters, whereas the included BMS watches that the defined limits of the battery are respected. In the end, the function returns the total amount of energy e_s that has been stored or discharged during this timeframe. For instance, if the battery was requested to charge at $p_s = 20W$ for $d_s = 10s$, but the BMS limits the charging to $15W$, the return value would be $e_s = 15W \cdot 10s = 150Ws$.

In addition to this basic charging and discharging, a **Storage** implementing the energy storage interface has to include functionality to allow for monitoring of the battery's internal state. As discussed in Section 2.3, the SoC is the key metric when it comes to determining the energy level of a battery, and therefore, there has to be a way for the BMS to give an estimation that can be used for energy management in the microgrid. For other important internal parameters that depend on the type of battery, the **Storage** can also define a custom state *state^s* that can be used for providing visibility over the battery system. The state is transmitted to the controllers via the Mosaik scheduler as described in Section 4.1.

4.3 Energy Management Policies

In the context of microgrid co-simulation, energy management is a critical part, as it has to be defined, how the microgrid responds to specific power deltas, which refer to the variations in power supply and demand. For this purpose, the approach allows for a user-defined **MicrogridPolicy**, which provides a structured way to deal with these power fluctuations, whereas the management includes interactions with the public grid and present energy storage systems as well as curtailment.

The policy lets users define custom rules and strategies that balance the power delta of the microgrid's generators and consumers at every time-step. In case of excess power during the time-step, energy has to be stored, fed back to the utility grid, or curtailed, while a lack of power has to be leveled using either grid energy or the energy received by discharging the energy storage. These rules are applied at every time-step based on the current power delta, the duration that the delta is valid for, and also the battery's SoC as computed by the **Storage**.

When the **MicrogridPolicy** is applied, it receives the current power delta Δp (see section 2.2.2) and the time d that passed since the last time-step. With the defined rules, the policy then decides, what portion of the power delta is to be balanced by charging or discharging the **Storage** that the policy can access. It then instructs the **Storage** to (dis-)charge with a power p_s for a specified duration d_s that should be equal to, or, in special cases, less than d .

As discussed in section 4.2, the **Storage** returns the actual energy e_s that could be charged or discharged given these inputs when it is updated. The policy finally uses that to compute the amount of energy e that the microgrid had to draw or that the microgrid could feed into the public utility grid during the time-step. The mentioned interactions between the **MicrogridPolicy** and the **Storage** are also depicted in Figure 4.2. In the most basic case, where no energy is curtailed, and the microgrid can exchange energy with the public grid at all times, the grid energy can be computed using the formula $e = \Delta p \cdot d - e_s$, where $\Delta p \cdot d$ marks the total energy surplus or deficit of the microgrid since the last time-step. If e was greater than zero, there was energy fed to the grid, and in the case of e being less than zero, energy had to be drawn from the grid.

The **MicrogridPolicy** separates itself from the BMS of the energy storage in the way that it can also handle energy flow from or to the public grid, and can be, for instance, used to model the operating mode (grid-connected or islanded as defined in section 2.1) of the microgrid. Because the **MicrogridPolicy** only has access to the abstract energy storage interface, it is also more general, as it does not depend on the battery model in comparison to the BMS.

Chapter 5

Battery Model Analysis

In this chapter, the implementation of four models for lithium-ion batteries is discussed, and their behavior in a co-simulation environment, depicting a data center's power system, is analyzed. Section 5.1 introduces and parameterizes the used battery models, Section 5.2 presents the experimental results of different scenarios, which are then discussed in Section 5.3.

5.1 Battery Models for Co-Simulation

The explained battery models are parameterized to represent a battery pack of rechargeable Lithium-Ion INR21700 M50 cells ¹. Figure 5.1 depicts an exemplary pack of 16 cells, where four series groups are connected in parallel (4S4P).

There are four different mathematical models described in total that increase in complexity. Section 5.1.1 discusses the simplest lossless model, Section 5.1.2 adds power constraints and inefficiencies, in Section 5.1.3 electrochemistry of individual cells are considered, and Section 5.1.4 extends this to include losses due to the battery pack's circuit on top.

5.1.1 SimpleBattery

The *SimpleBattery* only receives two parameters upon initialization, the battery's energy capacity C in Wh and the initial SoC, which is used for determining the battery's initial energy $b(0)$. When updated, given the power $p_s(t)$ and the duration $d_s(t)$ of that time-step t , the battery's new energy is then computed using this formula:

$$b(t) = \begin{cases} \min(C, b(t-1) + p_s(t) \cdot d_s(t)) & | p_s(t) \geq 0 \\ \max(0, b(t-1) + p_s(t) \cdot d_s(t)) & | p_s(t) < 0 \end{cases} \quad (5.1)$$

¹The product specification can be found under this link: <https://www.dnkpowers.com/wp-content/uploads/2019/02/LG-INR21700-M50-Datasheet.pdf>

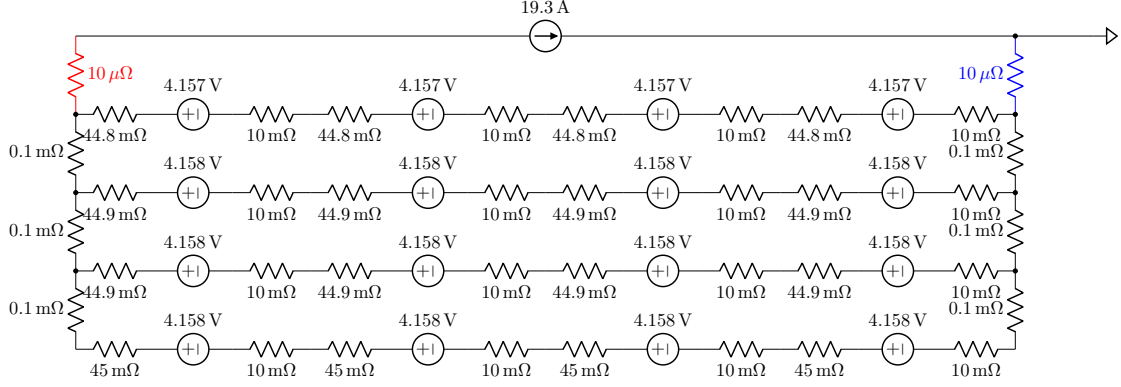


Figure 5.1: Visualization of a 4S4P battery pack, as obtained utilizing functions from the Liionpack [17] and Lcapy libraries [38] early during a discharging experiment. Voltage sources and a resistor to their left, indicating an internal resistance, represent the individual battery cells.

The (dis-)charged energy that is to be returned when implementing the proposed storage interface, is $p_s(t) \cdot d_s(t)$ in the default case, $-b(t-1)$ if the battery is fully discharged, or $C - b(t-1)$ if the battery is fully charged during the time-step. The formula $\text{SoC}(t) = \frac{b(t)}{C}$ determines the battery's state of charge.

The parameterization of this model to reflect the mentioned battery pack only requires a value for the battery capacity C . To ensure comparability between the different models, it was determined using a PyBaMM simulation of the SPMe model and the parameterization of the used cell by Chen et al. [39]. A complete discharge at $0.2C$ of a single cell returned an energy of $18.87Wh$ compared to the $18.20Wh$ that is listed in the product specification. This value is just multiplied by the number of cells to get the total energy capacity C of a battery pack.

5.1.2 CLCBattery

The *CLCBattery* implements the C-L-C model by Kazhamiaka et al. [15], which has been presented in detail in Section 2.4.1. When updated, the model behaves similar to the *SimpleBattery*, but includes constant inefficiency factors, so that the updated energy level can be computed based on the Equations 2.1 and 2.2:

$$b(t) = \begin{cases} b(t-1) + \eta_c \cdot p_s(t) \cdot d_s(t) & | p_s(t) \geq 0 \\ b(t-1) + \eta_d \cdot p_s(t) \cdot d_s(t) & | p_s(t) < 0 \end{cases} \quad (5.2)$$

The *CLCBattery* adds a lot of complexity regarding battery limitations, though. With a constant voltage V , as assumed in the model, and linear energy limits $a_1(I(t)) = u_1 \cdot I(t) + v_1$ and $a_2(I(t)) = u_2 \cdot I(t) + v_2$, the Equations 2.3, 2.4 and 2.5 result in the

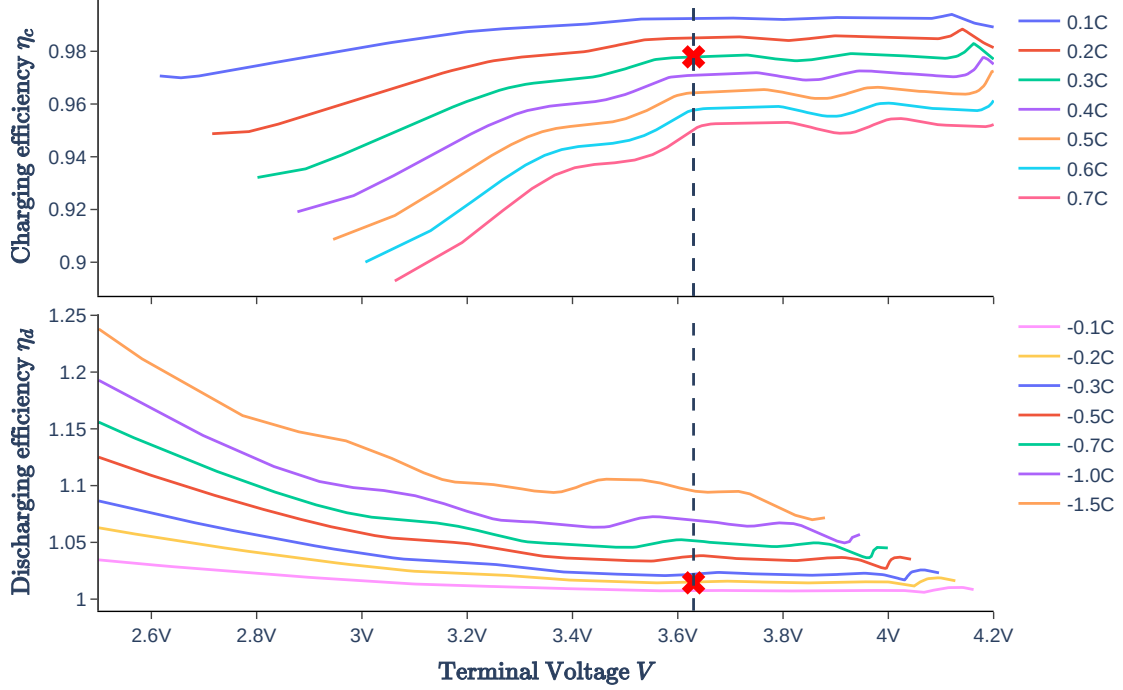


Figure 5.2: (Dis-)charging efficiencies based on battery voltage and (dis-)charging currents, as determined using PyBaMM.

following power limitations when charging or discharging at each time-step t :

$$\alpha_d \cdot V \leq p_s(t) \leq \alpha_c \cdot V \quad (5.3)$$

$$\frac{b(t-1) - v_1}{\frac{u_1}{V} - d_s(t) \cdot \eta_d} \leq p_s(t) \leq \frac{b(t-1) - v_2}{\frac{u_2}{V} - d_s(t) \cdot \eta_c} \quad (5.4)$$

These constraints can also be scaled linearly by the number of cells when considering battery packs.

Whenever the input power $p_s(t)$ would exceed these (dis-)charging limits, the BMS simply continues the (dis-)charge at the maximum allowed power. The total energy stored or discharged to be returned is therefore given by $e(t) = p_s(t) \cdot d_s(t)$. Similar to the *SimpleBattery*, the SoC can be easily computed as $\text{SoC}(t) = \frac{b(t)}{v_2}$.

The C-L-C model assumes constant Voltage $V(t)$, and, for the experiments, it is set to the nominal voltage of the modeled lithium-ion cell, which is $3.63V$ according to the product specification. The (dis-)charging current limits were defined to be $\alpha_d = -1.5C$ and $\alpha_c = 0.7C$ at $25^\circ C$ in the same way.

PyBaMM simulations, as shown in Figure 5.2, determined the constant inefficiency factors η_c and η_d . The charging efficiency is shown based on the voltage V and the charging rate, whereas the simulation computed the internal battery resistance R_i used

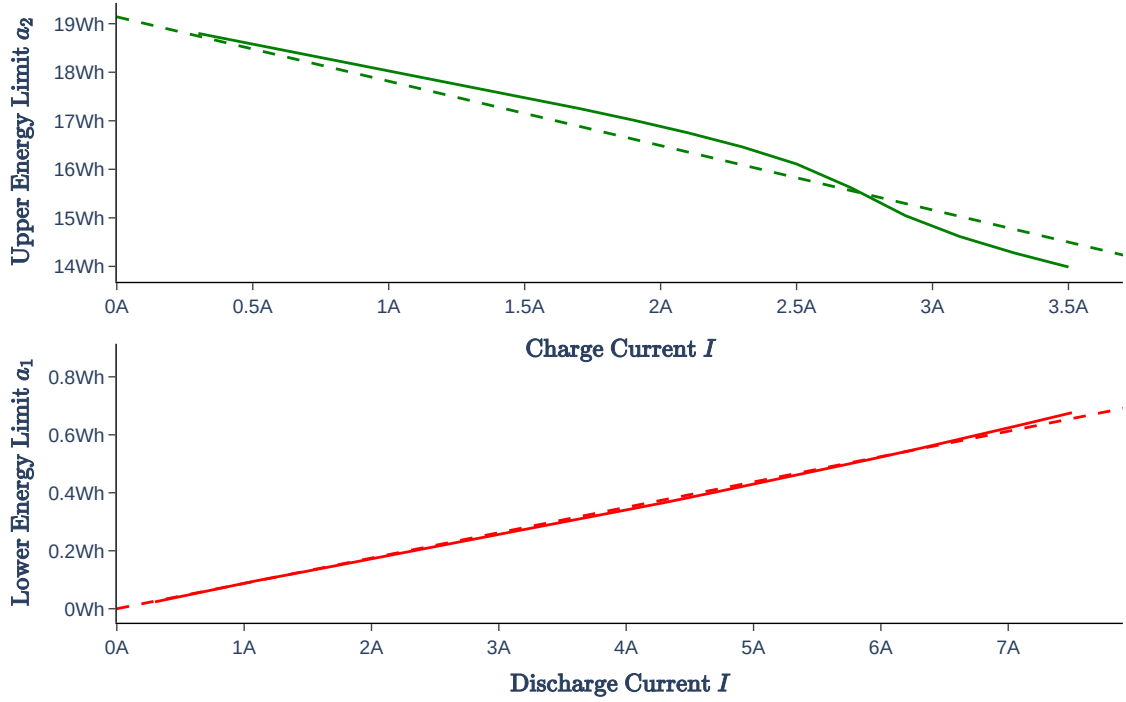


Figure 5.3: Energy limits of a single battery cell based on the (dis-)charging current.

to evaluate the formula described by Kazhamiaka et al. [15]:

$$\eta_c(I, V) = 1 - \frac{I \cdot R_i}{V} \quad (5.5)$$

The approximations for the inefficiencies are the values of these functions at the standard discharge rate of $0.2C$, and the standard charge rate of $0.3C$ at the nominal voltage of $3.63V$. The resulting inefficiencies for the cell described by the *CLCBattery* are therefore fixed at $\eta_d = 1.014$, and $\eta_c = 0.978$.

Finally, the linear approximations of the upper and lower energy limits, as shown in Figure 5.3, were again determined by PyBaMM simulations:

$$a_1(I) = -0.087 \cdot I, \quad a_2(I) = -1.326 \cdot I + 19.14$$

For battery packs, these energy limits can be scaled linearly by the number of individual cells as well.

5.1.3 PybammBattery

The *PybammBattery* uses the PyBaMM framework [16] directly to simulate an individual lithium-ion cell. For the experiments, the Single Particle Model with electrolyte, which was described in Section 2.4.2, is used together with an external circuit submodel to enable power-based inputs instead of current-based inputs. The single cell model

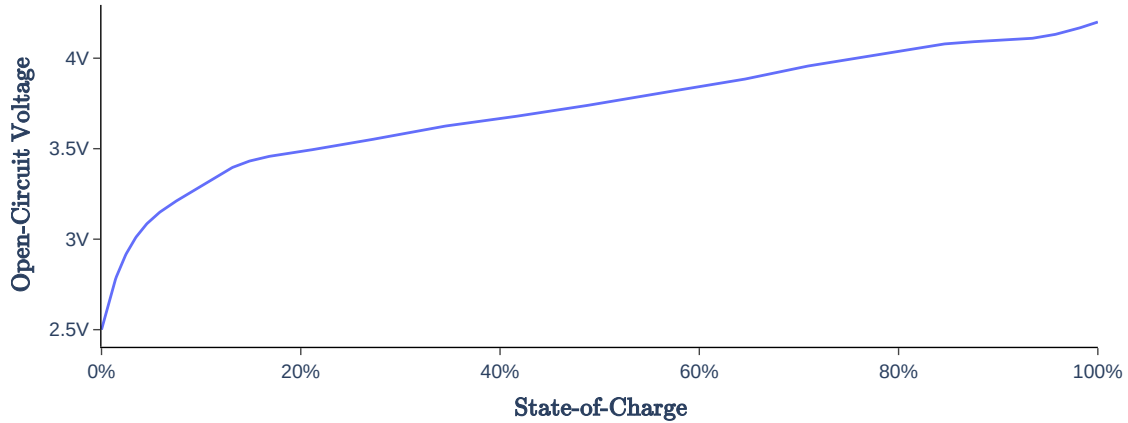


Figure 5.4: Relation between the battery cell’s State-of-Charge and Open-Circuit Voltage as obtained using PyBaMM.

utilizes the parameterization for the Lithium-Ion INR21700 M50 cell as experimentally obtained by Chen et al. [39]. To simulate battery packs, the input power $p_s(t)$ of time-step t is divided by the number of cells before the PyBaMM simulation is stepped.

The model is equipped with BMS functionality to ensure that battery cells do not exceed their limits, and visibility into the battery’s state is provided. Firstly, the battery’s Open-Circuit Voltage (OCV), which is the battery’s difference in potential when resting, is used together with an OCV–SoC lookup table, as visualized in Figure 5.4, to determine the SoC. This only offers a reasonable estimation of the SoC because the precise OCV is known in the model. In a real-world scenario, this approach is not feasible, as the OCV cannot be measured that accurately, and the relationship between the OCV and the SoC changes with temperature and battery age [40].

Secondly, also based on the OCV, power limits were determined to ensure that the battery does not exceed allowed currents and voltages when being (dis-)charged with a certain power. In particular, the currents cannot exceed $1.5C$, equivalent to $7.5A$, for discharging, and $0.7C$, equivalent to $3.5A$, for charging, whereas the battery cell’s voltage has to remain between $2.5V$ and $4.2V$. Two PyBaMM simulations determine the power limits depending on the OCV, with Figure 5.5 showing the results of the experiment. When the input power $p_s(t)$ would exceed the power limits based on the state, the maximum applicable power is used for (dis-)charging and the computation of the stored or discharged energy, which has to be returned.

5.1.4 LiionBatteryPack

The *LiionBatteryPack* utilizes PyBaMM [16] for simulation of battery cells, as well as methods for computing power-losses and solving power networks by Liionpack [17] to model an entire lithium-ion battery pack, such as the one visualized in Figure 5.1. It performs a series of steps to determine the new battery state at every time-step t :

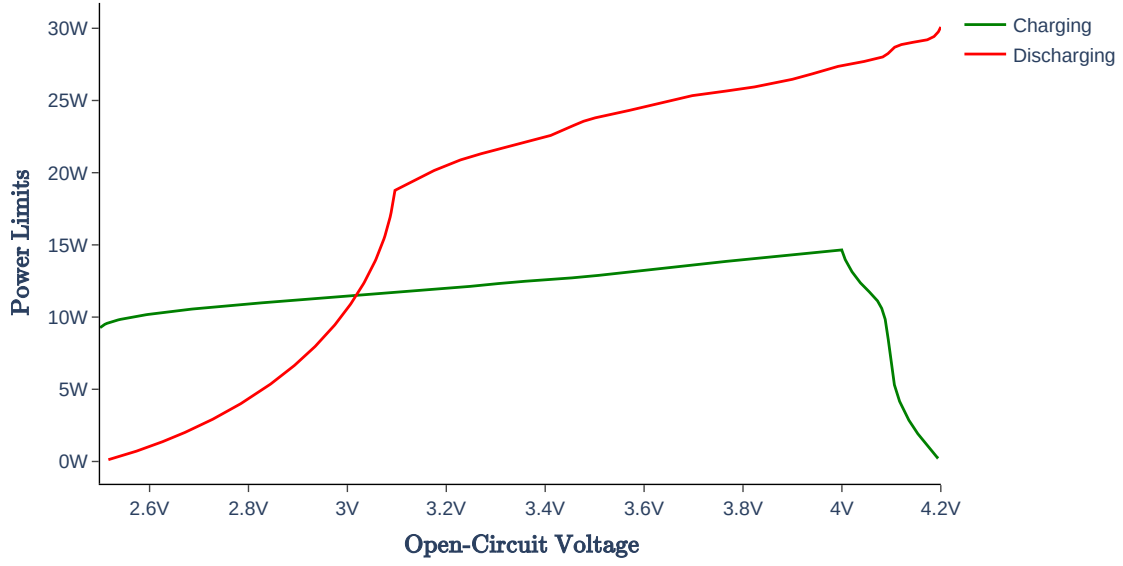


Figure 5.5: Power limits for charging and discharging based on the cell's OCV. In the more linear parts, the limits are the result of current boundaries, whereas voltage boundaries cause power limitations at a low SoC when discharging and at high SoC when charging.

1. The internal resistances and voltages are computed for every individual cell and included in the network representation.
2. The network is solved based on the input power $p_s(t)$, which determines the applied currents for the cells.
3. The simulation steps the electrochemical system of each cell with their respective current input using PyBaMM.

Again, a BMS wraps the model. It computes the SoC for all the cells in the same way as for the *PybammBattery*, and then averages them in the end to obtain the SoC of the battery pack. The power limits are also determined similar to the PyBaMM model described in Section 5.1.3, as the model's power limits are computed for the cell with the highest voltage when charging and for the cell with the lowest voltage when discharging, before being scaled by the number of batteries in the battery pack. The determined applied power is once again used to compute the returned (dis-)charged energy $e(t)$ by multiplication with the duration $d_s(t)$.

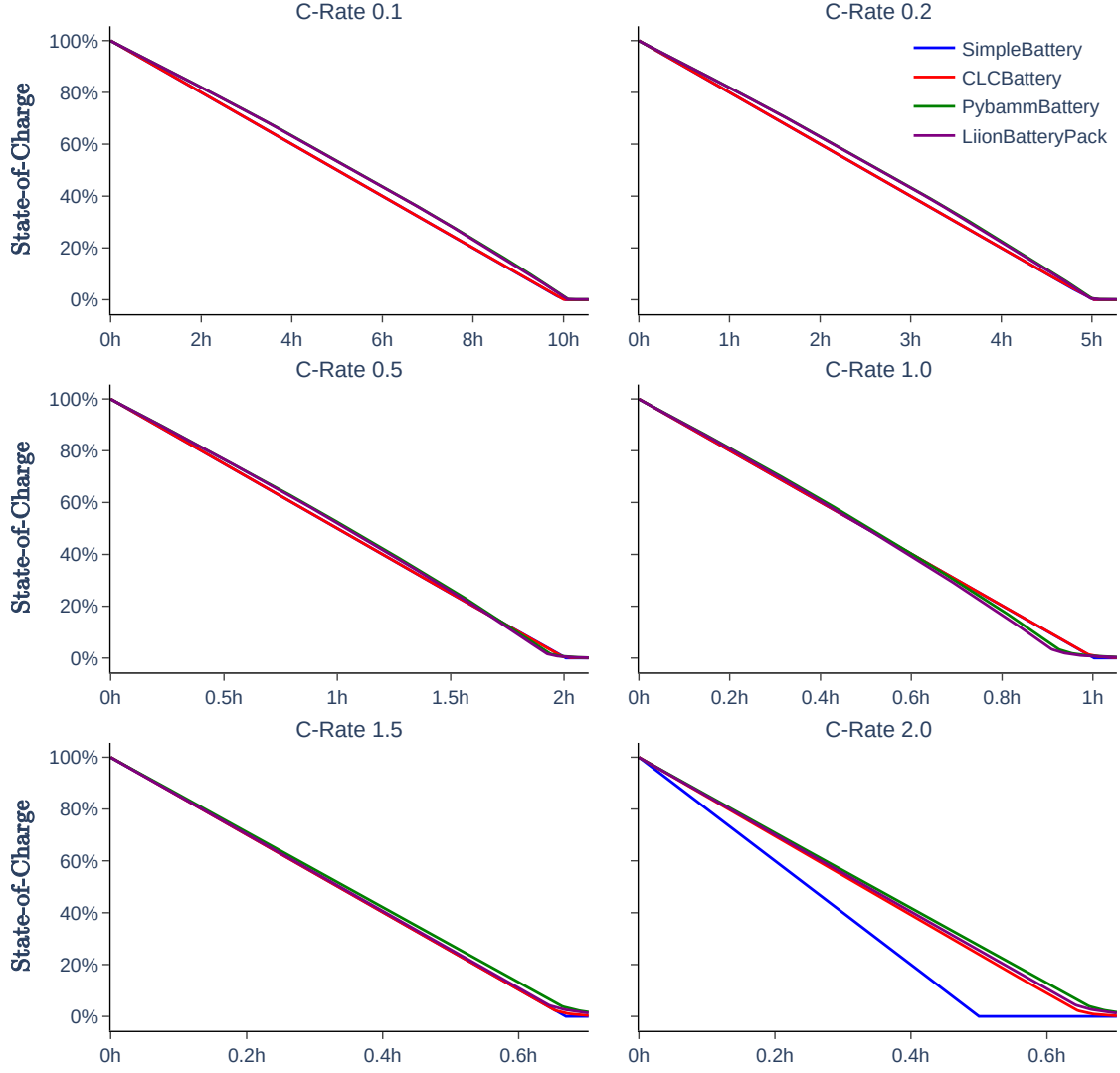


Figure 5.6: Behavior of the models when discharged at constant power with different rates.

5.2 Experimental Results

The described models are now used to represent a 16S16P battery pack of the INR21700 M50 cells for a series of experiments. Section 5.2.1 determines the behavior of the models in simulated scenarios of constant charging and discharging, Section 5.2.2 presents the results, when using the model in a simplified data center scenario, and Section 5.2.3 performs a runtime analysis.

5.2.1 Constant Power (Dis-)Charging

At first, it is shown, how the models behave when discharged at a constant power from a full to an empty state, and vice versa, whereas Figure 5.6 and Figure 5.7 visualize the results of the experiments.

When comparing the *SimpleBattery* to the *CLCBattery* in the discharging scenarios, it is noticeable that they produce almost the same traces, as both of their capacities were obtained using the drawn energy received by PyBaMM discharging experiments. Because the lower energy limits of the *CLCBattery* are not rising steeply, a real difference at low SoC values can only be seen at high discharge rates. As soon as these rates exceed $1.5C$, the *CLCBattery* limits the output power, resulting in the faster discharging of the *SimpleBattery*.

The *CLCBattery* achieves fairly accurate results relative to the *PybammBattery* and the *LiionBatteryPack* over all discharging rates. It is, however, visible that it empties slightly faster at low rates because the constant η_d factor overestimates the actual inefficiencies in these cases. The higher the applied power, the more the inefficiency is underestimated, which results in the slower discharges, when no current limitations are applicable. At $1.5C$, when limitations are in place, the *CLCBattery* once again empties faster than the complex models due to the voltage of their cells being lower here next to the nominal voltage the *CLCBattery* uses, resulting in smaller power limits.

The *PybammBattery* reflects the *LiionBatteryPack* almost perfectly, as only at higher discharging rates the power losses of the circuit, which are determined in the later model, have a high enough impact to be visible in the overall trend. Through this experiment, it can also be observed that in both of the models, the SoC is overestimated ever so slightly around half-full state.

The differences between the *SimpleBattery* and the *CLCBattery* become more obvious for the charging experiments. Even though they initially charge at the same speed at low to medium charging rates, due to the quickly falling upper energy limits of the *CLCBattery*, the *SimpleBattery* reaches its full state much sooner, as no power limitations are considered here. At charging rates exceeding the set $0.7C$, the BMS of the *CLCBattery* limits the current as well, resulting in an even slower discharge.

The *CLCBattery* seems to reflect the charging process of the *PybammBattery* and the *LiionBatteryPack* quite well again over all charging rates. Similar to the discharging experiment, the *CLCBattery* overestimates the inefficiency at low charging rates, and underestimates the inefficiency at higher charging rates. The underestimation is not as apparent in the figure due to aforementioned inaccuracies of the SoC estimation for the complex models.

Because the current limits, and the resulting power limits, are even lower for charging than for discharging, the differences between the *PybammBattery* and the *Liionpack* are even smaller in this experiment, as the power losses induced by the pack's circuit are almost negligible.

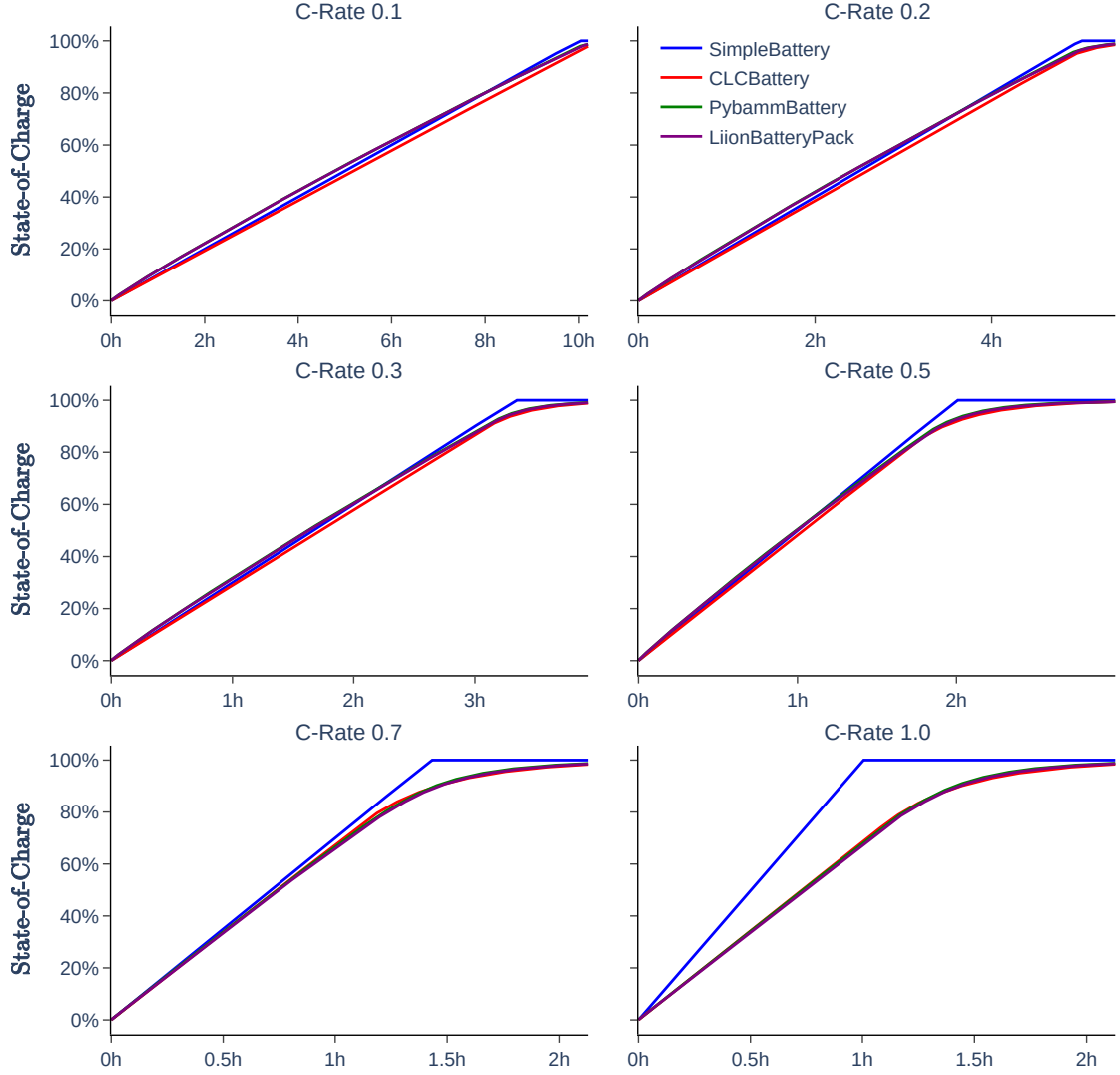


Figure 5.7: Behavior of the models when charged at constant power with different rates.

5.2.2 Data Center Scenario

While utilizing the enhanced co-simulation capabilities of Vessim [11], this section conducts an experiment over two days based on a more realistic system configuration that is supposed to represent the power system of a simplified data center. This is supposed to test the models in a more variable environment under real operating conditions. The scenario is similar to an experiment by Wiesner et al. [37] and contains two nodes, which represent servers in the data center, that have a constant power consumption of 200W and 50W respectively, which are aggregated and multiplied by a power usage effectiveness (PUE) of 1.3 to obtain the total power consumption of the mocked data

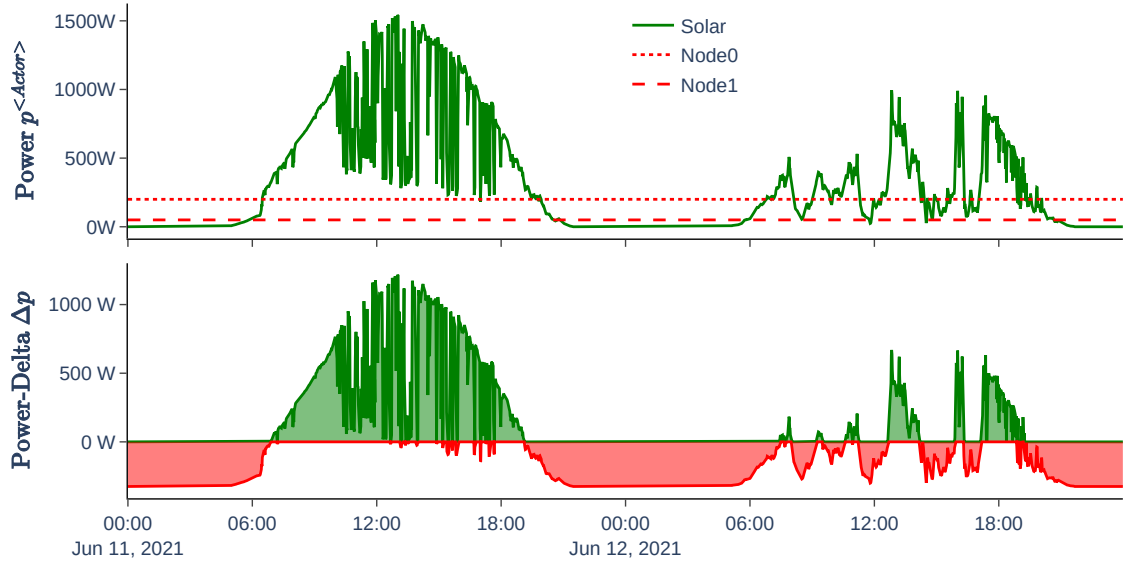


Figure 5.8: Power production, consumption, and power delta, as used in the experiment.

center. Additionally, the power production of a solar panel of $10m^2$ size with an efficiency of 15% is simulated, using data of global solar radiation in Berlin from June 2021². Figure 5.8 presents the resulting power traces.

In the scenario, the co-simulated microgrid is allowed to exchange energy with the public grid at all times, and the included **MicrogridPolicy** tries to (dis-)charge as much of the power-delta as possible using the **Storage**, while exchanging the remaining energy delta with the grid. On top, the policy allows a minimum SoC to be set, at which point it stops to further discharge the energy storage, and includes functionality to charge the **Storage** at a specific power rate using the power-delta and grid-energy.

During all the simulations, a custom controller is used that instructs the **Microgrid-Policy** to charge the battery pack at 3200W between 11AM and 12AM on the second day due to insufficient solar production. The minimum SoC is fixed to 30% at all times to ensure operation of the data center in case of grid outages.

The development of the SoC, depending on the model used, is represented in Figure 5.9, and the trends show the discovered characteristics outlined in Section 5.2.1. In general, while the *SimpleBattery* fails to accurately represent the SoC of the more complex models, in particular when charging at a high SoC, the *CLCBattery* captures the physical models quite well throughout the experiment. Again, no real differences in the estimation can be seen between the *PybammBattery* and the *LiionBatteryPack*.

When looking at the exchanged energy with the grid (depicted in Figure 5.10), it is obtained that over the course of the two days, the *LiionBatteryPack* used a net 1103Wh of energy from the grid, with the battery having to charge a total amount of 3435Wh,

²The data was retrieved from <https://wetter.htw-berlin.de>

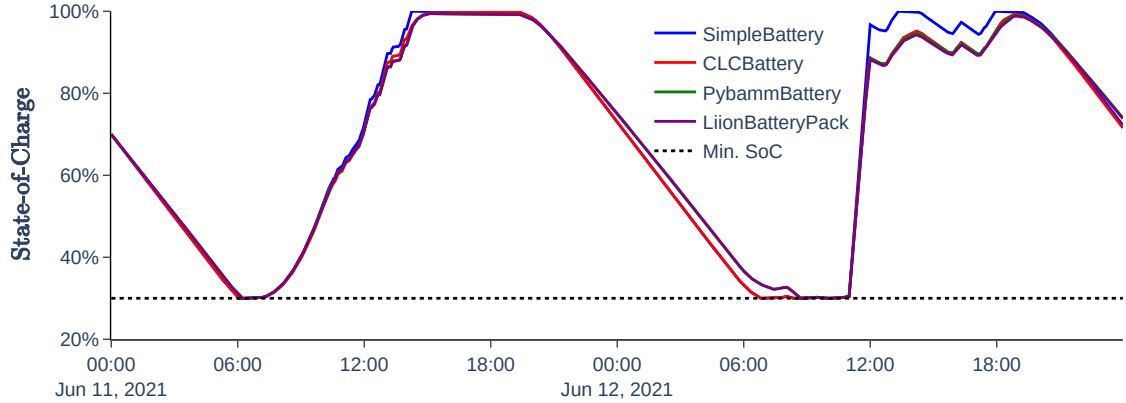


Figure 5.9: SoC Progression of the models during the experiment.

and having fed back only around $2332Wh$. This net grid charge is 1.9% higher than for the *PybammBattery* ($1081Wh$), 8.5% higher than for the *CLCBattery* ($1016Wh$), and a total of 41.5% higher than for the *SimpleBattery* ($779Wh$) for this particular scenario.

Because the *SimpleBattery* does not consider any inefficiencies, and is parameterized to best reflect the SoC of the more complex models, it charges and discharges completely using less energy so that more energy exchange with the grid is necessary. This is obviously not the case for the *CLCBattery*, hence the good approximation of the physical models. The remaining inaccuracy here comes from over- and underestimations of inefficiencies, and slightly different power limits, meaning that for a good parameterization of the C-L-C model, the difference in exchanged grid energy is likely to balance relative to the used *PybammBattery* over long experiments. When looking at the *PybammBattery* against the *LiionBatteryPack*, it can be determined that for the modeled battery pack, the power losses of the pack’s circuit do not make a huge difference in this particular experiment.

5.2.3 Runtime Analysis

To determine the efficiency and therefore the usefulness of the described models, a runtime analysis is conducted. A virtual node in the cloud ³ executed the runtime experiments for all models to ensure comparability between the obtained data. Figure 5.11 presents the experimental results.

As was to be expected, there is only little difference in performance between the *SimpleBattery* and the *CLCBattery* because both are linear models. The median execution time of a single simulation time-step for the *SimpleBattery* was around $191\mu s$ compared to $196\mu s$ of the *CLCBattery*. For both these models and the *PybammBattery*, the execution time does not change with the size of the battery pack, as only a single cell modeling approach is used, where relevant parameters are scaled using the number

³Machine type: Google Cloud Platform GCE C3

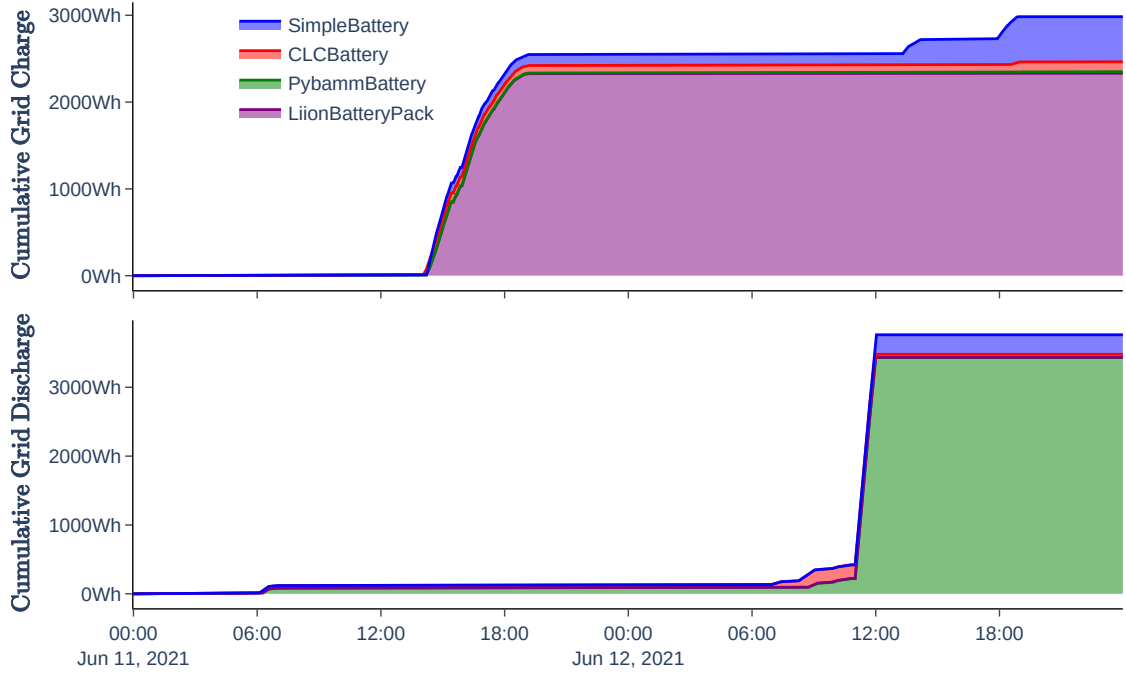


Figure 5.10: Exchanged power with the grid over the experiment. Grid discharge indicates energy used, while grid charge refers to energy fed to the utility grid.

of cells in the battery pack.

The complexity of the optimization problem for the *LiionBatteryPack* scales linearly with the number of cells, as can be seen in the figure. A $32S32P$ battery pack imitated using the *LiionBatteryPack* model with a total of 1024 cells can only step two to three times per second on the used node, whereas the model for the $16S16P$ pack of earlier experiments has a median execution time of $0.095s$. That is nearly 500 times higher than for the *SimpleBattery* and the *CLCBattery*, and more than 12 times higher than for the *PybammBattery* with $7.65ms$.

5.3 Discussion

Firstly, it can be said that the extended Vessim co-simulation architecture with the refined storage simulation unit described in Chapter 4 seamlessly allows the integration of various battery models. According to the results from the data center scenario, the proposed extension can be used to study energy- and battery management inside isolated power systems like microgrids, and the resulting architecture can therefore serve as a co-simulation testbed for data centers.

When we compare the models, it becomes obvious that for the modeled lithium-ion cell, the *SimpleBattery* fails to accurately represent the battery's SoC, especially for

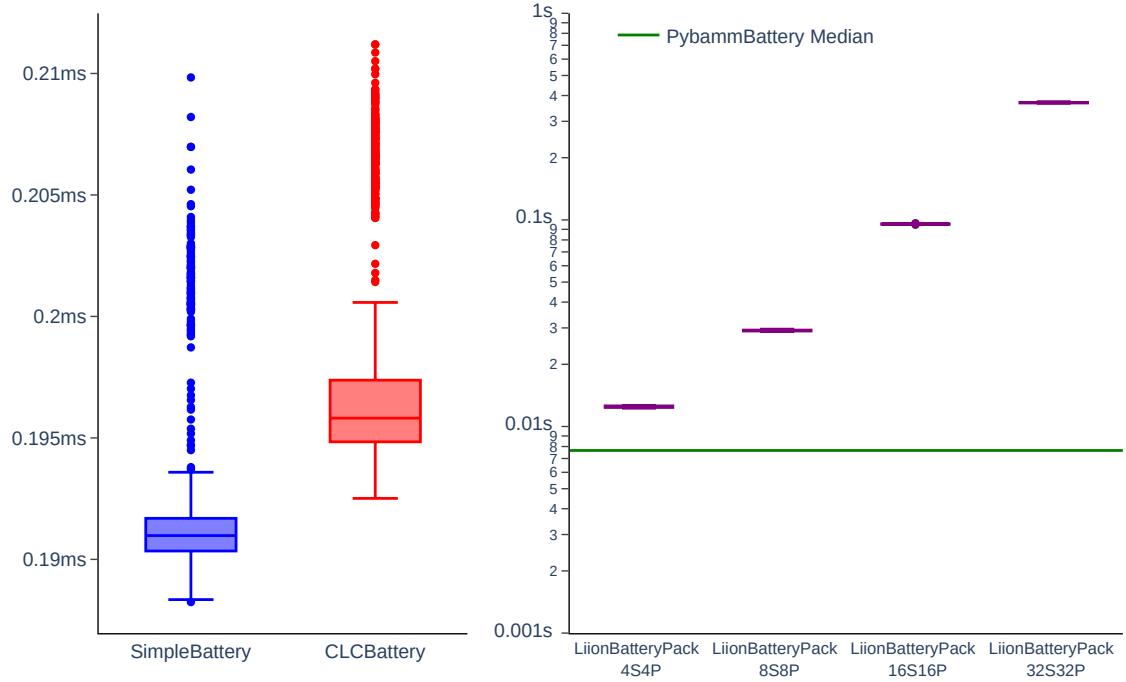


Figure 5.11: Runtime distributions when executing a single simulation time-step.

charging at an almost full state, while also providing inaccurate estimations of exchanged grid energy in a microgrid scenario. This makes it very unsuitable for experiments with the main focus on energy management. However, the *SimpleBattery* manages to capture the overall trends, which makes it still useful for scenarios that only try to determine broad tendencies like, for instance, the effectiveness of load forecasting. The reason for that is the straightforward parameterization, the fast runtime, and the independence of the used energy storage, as not only batteries can be modeled this way.

Compared to the *SimpleBattery*, the *CLCBattery* has only minimal performance overhead, while being able to estimate a battery’s state, and used grid energy well in comparison to physics-based models. The experiments indicate that the C-L-C model with a decent parameterization has the potential to accurately represent a lithium-ion cell’s behavior in short-term experiments. It is therefore applicable for most use-cases where a microgrid is simulated. A representative parameterization for the utilized battery cell can be easily found using product specifications or other experimental data, and even tools for battery simulation like PyBaMM [16].

Even though SPMs are considered to be the simplest electrochemical models, as Section 2.4.2 outlines, solving such models still comes with a big computational cost. On top, a decent parameterization can only be obtained through real-world experiments, although frameworks like PyBaMM provide them for a few cell types, and there are difficulties in obtaining a useful estimation of the cell’s SoC. There is also uncertainty

whether an SPM or other electrochemical models actually represent a real battery cell accurately. These types of models are therefore not useful for most short-term scenarios involving co-simulated microgrids, as these representations of power systems possibly bring additional inaccuracies, so that the usage of physics-based models is not justifiable. There exists, however, the opportunity to utilize such electrochemical models in long-term experiments, when aspects like battery degradation in data center scenarios are of interest, as models such as SPMs can be extended with coupled degradation mechanisms as investigated by O’Kane et al. [41].

For the rather small battery packs investigated in this thesis, there does not seem to be a payoff in modeling a whole battery pack instead of a single cell that is scaled up, as the discovered effects of power losses were fairly minimal. Because the parameterization of the *LiionBatteryPack*’s circuit was not modeled after a real battery pack, the results in that regard might not be reliable, however. For a larger battery pack, the execution times become too substantial, even up to the point where it would be critical for real-time applications, but the stepping of the electrochemical battery cells could potentially run in parallel to solve this runtime issue. All in all, the results of the experiments using the *LiionBatteryPack* have the most uncertainty to them, yet the collected data questions the usefulness of modeling a battery pack in a way that considers electrochemical cells and the pack’s circuit.

Chapter 6

Conclusion

This thesis introduced an extension to the Vessim framework [11], an already existing co-simulation testbed for carbon-aware applications and systems, to allow for better simulations of battery packs, depending on the experimentation context. The extension included the definition of an energy storage interface, allowing the implementation of models for battery systems of different complexity, and established policies to be used for energy management during the simulation. Additionally, the improved architecture allowed *visibility* and *control* over the energy storage system through controllers and message passing, using the underlying scheduler of Mosaik [14].

To evaluate the usefulness of utilizing various models for simulating lithium-ion battery packs, this work conducted an analysis, integrating a total of four models into the described framework. The first model described a simple, lossless battery that did not consider any power limitations, whereas the second implementation was based on the C-L-C model by Kazhamiaka et al. [15], adding inefficiencies and power constraints. The third model made use of the PyBaMM framework [16] to simulate an individual cell using the physics-based Single Particle Model, which was scaled to represent a battery pack, before the final model applied multiple of these electrochemical models together with an external circuit to reproduce the behavior of a realistic battery pack.

The experiments showed that, despite its abilities to determine overall trends for a battery's state and exchanged grid energy, the lack of accuracy by the simplest model severely limited its usability in experiments studying energy management of computing infrastructure. Adding linear constraints and inefficiencies with the C-L-C model massively improved such estimations with only minimal performance overhead, making those kinds of models ideal for most scenarios, due to its simple process of parameterization.

Although SPMs offered a more accurate insight into the electrochemical processes of battery cells, the high computational cost for stepping the models and the complexities to determine an accurate parameterization made them impractical for short-term scenarios. They could, however, still be used to study long-term effects like battery degradation, although further research would have to be conducted. Modeling a whole

battery pack instead of a single scaled-up cell was found to be inefficient, as only minor power-loss effects are noticeable for small battery packs, and larger battery packs come with significant execution times, raising questions about the utility of such detailed modeling.

To extend on this thesis, future work should focus on benchmarking the used simulation tools against real systems. For instance, there is still much uncertainty, whether co-simulation testbeds such as Vessim accurately determine the power-deltas that are used to (dis-)charge the energy storage components compared to real-world power systems. On top, there is a need to evaluate models like the SPMs by PyBaMM, and a parameterization of an individual cell against real-world data to ensure accurate long-term behavior. Kazhamiaka et al. [15] also described other tractable models for lithium-ion battery cells with more advanced approximations like the C-L-L or the L-L-Q model to be investigated in the same ways as described in this thesis.

Bibliography

- [1] P. Dhar, “The carbon impact of artificial intelligence,” *Nat. Mach. Intell.*, vol. 2, no. 8, pp. 423–425, 2020.
- [2] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, “Recalibrating global data center energy-use estimates,” *Science*, vol. 367, no. 6481, pp. 984–986, 2020.
- [3] J. K. Boyce, “Carbon pricing: effectiveness and equity,” *Ecological Economics*, vol. 150, pp. 52–61, 2018.
- [4] R. Best, P. J. Burke, and F. Jotzo, “Carbon pricing efficacy: Cross-country evidence,” *Environmental and Resource Economics*, vol. 77, no. 1, pp. 69–94, 2020.
- [5] W. Li, T. Yang, F. C. Delicato, P. F. Pires, Z. Tari, S. U. Khan, and A. Y. Zomaya, “On enabling sustainable edge computing with renewable energy resources,” *IEEE Communications Magazine*, vol. 56, no. 5, pp. 94–101, 2018.
- [6] A. Banerji, D. Sen, A. K. Bera, D. Ray, D. Paul, A. Bhakat, and S. K. Biswas, “Microgrid: A review,” in *2013 IEEE Global Humanitarian Technology Conference: South Asia Satellite (GHTC-SAS)*, pp. 27–35, 2013.
- [7] R. Lasseter, “Microgrids,” in *2002 IEEE Power Engineering Society Winter Meeting. Conference Proceedings (Cat. No.02CH37309)*, vol. 1, pp. 305–308 vol.1, 2002.
- [8] M. H. Saeed, W. Fangzong, B. A. Kalwar, and S. Iqbal, “A review on microgrids’ challenges & perspectives,” *IEEE Access*, vol. 9, pp. 166502–166517, 2021.
- [9] X. Tan, Q. Li, and H. Wang, “Advances and trends of energy storage technology in microgrid,” *International Journal of Electrical Power & Energy Systems*, vol. 44, no. 1, pp. 179–191, 2013.
- [10] L. Yu, T. Jiang, and Y. Zou, “Distributed real-time energy management in data center microgrids,” *IEEE Transactions on Smart Grid*, vol. 9, no. 4, pp. 3748–3762, 2016.
- [11] P. Wiesner, I. Behnke, P. Kilian, M. Steinke, and O. Kao, “Vessim: A testbed for carbon-aware applications and systems,” in *3rd Workshop on Sustainable Computer Systems (HotCarbon)*, 2024.
- [12] C. Gomes, C. Thule, D. Broman, P. G. Larsen, and H. Vangheluwe, “Co-simulation: a survey,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–33, 2018.

- [13] A. Alzahrani, M. Ferdowsi, P. Shamsi, and C. H. Dagli, “Modeling and simulation of microgrid,” *Procedia Computer Science*, vol. 114, pp. 392–400, 2017.
- [14] C. Steinbrink, M. Blank-Babazadeh, A. El-Ama, S. Holly, B. Lüers, M. Nebel-Wenner, R. P. Ramírez Acosta, T. Raub, J. S. Schwarz, S. Stark, A. Nieße, and S. Lehnhoff, “Cpes testing with mosaik: Co-simulation planning, execution and analysis,” *Applied Sciences*, vol. 9, no. 5, 2019.
- [15] F. Kazhamiaka, C. Rosenberg, and S. Keshav, “Tractable lithium-ion storage models for optimizing energy systems,” *Energy Informatics*, vol. 2, pp. 1–22, 2019.
- [16] V. Sulzer, S. G. Marquis, R. Timms, M. Robinson, and S. J. Chapman, “Python battery mathematical modelling (pybamm),” *Journal of Open Research Software*, vol. 9, no. 1, 2021.
- [17] T. Tranter, R. Timms, V. Sulzer, F. Planella, G. Wiggins, S. Karra, P. Agarwal, S. Chopra, S. Allu, P. Shearing, *et al.*, “liionpack: A python package for simulating packs of batteries with pybamm,” *Journal of Open Source Software*, vol. 7, no. 70, 2022.
- [18] R. Lasseter and P. Paigi, “Microgrid: a conceptual solution,” in *2004 IEEE 35th Annual Power Electronics Specialists Conference (IEEE Cat. No.04CH37551)*, vol. 6, pp. 4285–4290 Vol.6, 2004.
- [19] C. Marnay, S. Chatzivasileiadis, C. Abbey, R. Iravani, G. Joos, P. Lombardi, P. Mancarella, and J. von Appen, “Microgrid evolution roadmap,” in *2015 International Symposium on Smart Electric Distribution Systems and Technologies (EDST)*, pp. 139–144, 2015.
- [20] R. Bayindir, E. Hossain, E. Kabalci, and R. Perez, “A comprehensive study on microgrid technology,” *International Journal of Renewable Energy Research*, vol. 4, no. 4, pp. 1094–1107, 2014.
- [21] G. Schweiger, C. Gomes, G. Engel, I. Hafner, J. Schoeggl, A. Posch, and T. Nouidui, “An empirical survey on co-simulation: Promising standards, challenges and research needs,” *Simulation modelling practice and theory*, vol. 95, pp. 148–163, 2019.
- [22] X. Fang, S. Misra, G. Xue, and D. Yang, “Smart grid — the new and improved power grid: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 14, no. 4, pp. 944–980, 2012.
- [23] T. Brown, J. Hörsch, and D. Schlachtberger, “Pypsa: Python for power system analysis,” *arXiv preprint arXiv:1707.09913*, 2017.
- [24] H. Rahimi-Eichi, U. Ojha, F. Baronti, and M.-Y. Chow, “Battery management system: An overview of its application in the smart grid and electric vehicles,” *IEEE industrial electronics magazine*, vol. 7, no. 2, pp. 4–16, 2013.
- [25] H. J. Bergveld, W. S. Kruijt, P. H. Notten, H. J. Bergveld, W. S. Kruijt, and P. H. Notten, *Battery management systems*. Springer, 2002.

- [26] T. Kim, W. Song, D.-Y. Son, L. K. Ono, and Y. Qi, “Lithium-ion batteries: outlook on present, future, and hybridized technologies,” *Journal of materials chemistry A*, vol. 7, no. 7, pp. 2942–2964, 2019.
- [27] D. Deng, “Li-ion batteries: basics, progress, and challenges,” *Energy Science & Engineering*, vol. 3, no. 5, pp. 385–418, 2015.
- [28] F. Kazhamiaka, S. Keshav, C. Rosenberg, and K.-H. Pettinger, “Simple spec-based modeling of lithium-ion batteries,” *IEEE Transactions on Energy Conversion*, vol. 33, no. 4, pp. 1757–1765, 2018.
- [29] B. Acun, B. Lee, F. Kazhamiaka, K. Maeng, M. Chakkaravarthy, U. Gupta, D. Brooks, and C.-J. Wu, “Carbon explorer: A holistic approach for designing carbon aware datacenters,” *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023.
- [30] M. Doyle, T. F. Fuller, and J. Newman, “Modeling of galvanostatic charge and discharge of the lithium/polymer/insertion cell,” *Journal of the Electrochemical society*, vol. 140, no. 6, p. 1526, 1993.
- [31] S. G. Marquis, V. Sulzer, R. Timms, C. P. Please, and S. J. Chapman, “An asymptotic derivation of a single particle model with electrolyte,” *Journal of The Electrochemical Society*, vol. 166, no. 15, p. A3693, 2019.
- [32] F. B. Planella, M. Sheikh, and W. D. Widanage, “Systematic derivation and validation of a reduced thermal-electrochemical model for lithium-ion batteries using asymptotic methods,” *Electrochimica Acta*, vol. 388, p. 138524, 2021.
- [33] V. Ramadesigan, P. W. Northrop, S. De, S. Santhanagopalan, R. D. Braatz, and V. R. Subramanian, “Modeling and simulation of lithium-ion batteries from a systems engineering perspective,” *Journal of the electrochemical society*, vol. 159, no. 3, p. R31, 2012.
- [34] J. Li, N. Lotfi, R. G. Landers, and J. Park, “A single particle model for lithium-ion batteries with electrolyte and stress-enhanced diffusion physics,” *Journal of The Electrochemical Society*, vol. 164, no. 4, p. A874, 2017.
- [35] V. Kounev, D. Tipper, M. Levesque, B. M. Grainger, T. Mcdermott, and G. F. Reed, “A microgrid co-simulation framework,” in *2015 Workshop on Modeling and Simulation of Cyber-Physical Energy Systems (MSCPES)*, pp. 1–6, 2015.
- [36] O. Abrishambaf, P. Faria, L. Gomes, J. Spínola, Z. Vale, and J. M. Corchado, “Implementation of a real-time microgrid simulation platform based on centralized and distributed management,” *Energies*, vol. 10, no. 6, p. 806, 2017.
- [37] P. Wiesner, M. Steinke, H. Nickel, Y. Kitana, and O. Kao, “Software-in-the-loop simulation for developing and testing carbon-aware applications,” *Software: Practice and Experience*, vol. 53, no. 12, pp. 2362–2376, 2023.
- [38] M. Hayes, “Lcapy: symbolic linear circuit analysis with Python,” *PeerJ Computer Science*, p. e875, Feb. 2022.

- [39] C.-H. Chen, F. B. Planella, K. O'regan, D. Gastol, W. D. Widanage, and E. Kendrick, "Development of experimental techniques for parameterization of multi-scale lithium-ion battery models," *Journal of The Electrochemical Society*, vol. 167, no. 8, p. 080534, 2020.
- [40] J. Meng, M. Ricco, G. Luo, M. Swierczynski, D.-I. Stroe, A.-I. Stroe, and R. Teodorescu, "An overview and comparison of online implementable soc estimation methods for lithium-ion battery," *IEEE Transactions on Industry Applications*, vol. 54, no. 2, pp. 1583–1591, 2017.
- [41] S. E. O'Kane, W. Ai, G. Madabattula, D. Alonso-Alvarez, R. Timms, V. Sulzer, J. S. Edge, B. Wu, G. J. Offer, and M. Marinescu, "Lithium-ion battery degradation: how to model it," *Physical Chemistry Chemical Physics*, vol. 24, no. 13, pp. 7909–7922, 2022.